COMPUTER SCIENCE + COMPUTER ENGINEERING FOR GAMES

09

VOLUME NINE

PACKAGE 10 / GRADUATE SPECIALIZATION

Graduate Game-Engine and Graphics Engineering

Expanded teaching text with derivations, worked examples and original quizzes

20 LESSONS / MODULES 41-45 / GATE I

Edition: September 28, 2026

Original lesson sequence preserved. Self-study materials and runnable educational references; not an accredited degree or completed learner submission.

Contents

How this expansion fits the program 4

Reading the lesson layout . . . 4

Assessment and evidence . . . 4

Citation and version policy . . . 4

Mathematical notation . . . 5

Module 41 — Graduate numerical methods and research practice 6

Lesson 41.1 — Reading research as claims, assumptions and evidence . . . 7

Lesson 41.2 — Conditioning, sparse systems and iterative solvers . . . 9

Lesson 41.3 — Constrained optimization, derivatives and optimality conditions . . . 11

Lesson 41.4 — Experimental design, uncertainty and reproducible graduate work . . . 13

Module 41 project: numerical research notebook . . . 15

Module 42 — Advanced parallel and GPU algorithms 16

Lesson 42.1 — Parallel reductions, scans and work-span analysis . . . 17

Lesson 42.2 — Tiling, data reuse and asynchronous pipelines . . . 19

Lesson 42.3 — Irregular workloads, wavefront scheduling and load balance . . . 21

Lesson 42.4 — Portable performance and integration-quality evaluation . . . 23

Module 42 project: reusable GPU primitive in an engine . . . 25

Module 43 — Advanced rendering and reconstruction 26

Lesson 43.1 — Monte Carlo light transport and multiple importance sampling . . . 27

Lesson 43.2 — Real-time ray tracing, dynamic acceleration and hybrid pipelines . . . 30

Lesson 43.3 — Spatiotemporal reuse, reservoirs and denoising . . . 32

Lesson 43.4 — Differentiable and neural rendering under engineering constraints . . . 34

Module 43 project: bounded rendering reproduction . . . 36

Module 44 — Advanced engine systems and simulation research 37

Lesson 44.1 — Large-world scheduling, streaming and critical-path control . . . 38

Lesson 44.2 — Deformable simulation, variational models and solver tradeoffs . . . 40

Lesson 44.3 — Distributed game state, consistency and overload behavior . . . 43

Lesson 44.4 — Formal specifications, replay and reliability-oriented engine design . . . 45

Module 44 project: one engine-system depth experiment . . . 47

Module 45 — Compilers, shader languages and engine specialization 48

Lesson 45.1 — Intermediate representations, SSA and dataflow analysis . . . 49

Lesson 45.2 — Alias analysis, vectorization and legal optimization . . . 51

Lesson 45.3 — Domain-specific languages, specialization and safe JIT execution . . . 53

Lesson 45.4 — Compiler/hardware co-design and graduate artifact review . . . 55

Module 45 project: compiler mini-project plus Gate I . . . 57

Primary reading directory 58

How this expansion fits the program

This handbook expands the original Volume 9 teaching primers while preserving the manifest's 20 lessons, their learning outcomes, original lab briefs and three-question quizzes. Expanded sections add derivations, engineering contracts, worked failure cases and focused primary reading. The separate workbook contains 20 procedures and 40 additional graduate problems. Solutions remain in ANSWER_KEY.pdf and closed answer panels in the browser edition.

Use START_NEXT.pdf for prerequisites, environment and the first session. Planning remains 550-850 hours including assignments and overlapping Gate I work. The practical standard is an explained, tested artifact, not completion by reading. Research and engineering conclusions must remain no broader than the evidence.

Reading the lesson layout

Each lesson begins with the original outcome and primer, then develops the mathematics or system contract. The expanded worked example shows a concrete calculation or failure history. The original lab brief remains visible beside a more detailed procedure. Complete the assigned external study, implement the lab, answer the quiz and solve both graduate problems before marking the lesson complete.

The code references supply runnable oracles and diagnostic experiments, not finished student submissions. Twenty separate warm-up files intentionally raise NotImplementedError. Their visible cases help begin implementation; passing one is not proof of the complete graduate lab. Reference source should be consulted after an honest independent attempt and used to investigate discrepancies.

Assessment and evidence

Formative checks are corrected until all three quiz answers can be explained, both graduate problems are solved, and a fresh transfer case succeeds. Each module has an integrated project and a suggested 100-point rubric. Gate I additionally requires a bounded reproduction plus extension or rigorous failure analysis, an ablation, sensitivity, raw data and independent review. Passing the bundled 64-test reference suite is a package check, not a course completion certificate.

The module projects can feed one coherent Gate I direction. The compiler mini-project is required even when the main research topic is rendering or simulation. Actual GPU evidence remains pending until run on a supported device; analytical CPU scheduling models do not replace it. Scope statements in code/README.md and evidence/README.md are part of the scientific contract.

Citation and version policy

Original R reference IDs are preserved. S references are focused primary additions. REFERENCE_GUIDE.pdf gives their full titles, URLs and retrieval limits. Historical papers are selected as reproducible learning targets, not advertised as the latest techniques. Moving compiler, engine and API documentation is a reading reference, not proof that a particular installed toolchain is compatible. Record the exact versions actually tested in every learner artifact.

Mathematical notation

Plain-text formulas use * for multiplication, ^ for powers, ||x|| for a norm, grad for a gradient, delta for an increment and h or dt for a timestep. Vectors and matrices are named explicitly in each worked case. Norms, units, density measures and arithmetic widths must be stated where they affect a conclusion. A convenient-looking formula without its assumptions is not a complete method.

Module 41 — Graduate numerical methods and research practice

Prerequisites: Gates B, F and H; rigorous linear algebra, calculus, probability and systems competence.

Original project brief: Produce a graduate numerical notebook with derivations, reference implementations, conditioning experiments and a preregistered evaluation plan. Complete substantial assigned problem sets; the short lesson quizzes are only formative checks.

Core reference IDs: R42, R54, R09, R23. Full entries are in the reading directory.

- 41.1 — Reading research as claims, assumptions and evidence
- 41.2 — Conditioning, sparse systems and iterative solvers
- 41.3 — Constrained optimization, derivatives and optimality conditions
- 41.4 — Experimental design, uncertainty and reproducible graduate work

Lesson 41.1 — Reading research as claims, assumptions and evidence

Learning outcome: Extract an implementable argument rather than only a paper summary.

Planning: 18-26 hours. Includes assigned study and practical work; module/Gate I time overlaps.

Original teaching primer

A research paper makes claims under assumptions using a method and evidence. First identify the problem, baseline, objective and scope. Then reconstruct the central derivation or algorithm without looking at the authors' code. Track definitions, units and where approximations enter. A result can be valuable while narrower than its abstract initially suggests. Build a claim-evidence table: each claim points to an equation, proof, experiment or stated limitation. Distinguish reproduction of reported results from an independent implementation and from a new extension. Read related work to identify the actual contribution. Select original papers and author artifacts where available, and record versions. A negative reproduction result needs diagnostic investigation before concluding the original claim is false.

Turn a paper into an engineering contract

Write the question as an input-output relationship, not a topic: "For this scene family and error threshold, does method A reach the threshold in less total time than baseline B?" Specify scene motion, resolution, light/material model, numerical precision and the reference used to estimate error. The claim is a predicate over those conditions. An attractive image is an observation; it is not yet the predicate's proof. A reproducible implementation needs enough definitions that a second engineer can construct the same input and decide whether the output passes.

Separate three objects in your notes. The mathematical method defines ideal behavior. The implementation chooses finite precision, data layouts and scheduling. The experiment samples particular workloads on particular machines. A failure in one object does not automatically falsify the other two. For example, an unbiased estimator can be implemented with a wrong density; an accurate implementation can be compared with the wrong quality target. The assigned original papers supply the methods; this contract structure is the course's review procedure. [R44, S05]

Reconstruct before consulting implementation details

On the first pass, collect symbols, domains and units. On the second, derive the central update using a case small enough to calculate by hand. On the third, identify each approximation and each missing engineering choice. Do not hide an omitted detail with a convenient default: record it as an ambiguity and test whether plausible choices change the result. Only then compare your interpretation with author code, supplements or errata.

A claim-evidence map should record the exact figure, equation or artifact supporting each claim; the controlled comparison; one assumption that could break it; and your proposed test. Keep the original claim beside your narrower reproduction claim. This prevents a one-dimensional compliance experiment from becoming a claim to have reproduced a complete cloth benchmark.

Choose a tractable graduate contribution

A useful extension changes one dimension: a new workload distribution, a stricter correctness contract, a memory budget, a failure mode or an algorithmic component. Merely porting code is a contribution to an artifact, but does not by itself demonstrate a new scientific result. Conversely, a rigorous negative result can be valuable when it exposes a boundary and eliminates implementation error as the explanation.

Declare which stage is complete: method reconstruction, result reproduction, extension evaluation or independent review. These labels are deliberately explicit because communities use "reproducibility" and "replicability" differently. Passing the bundled smoke tests establishes none of the later stages on its own. [S14]

Worked engineering example

Suppose a baseline has RMSE 0.020 at 4 ms and 0.010 at 16 ms. A candidate has RMSE 0.012 at 4 ms and 0.009 at 8 ms. At the predeclared threshold RMSE <= 0.010, the measured qualifying times are 16 and 8 ms, giving 2x speedup among these tested configurations. At equal 4 ms cost, the candidate has 40% lower RMSE instead. Neither statement establishes its time at exactly 0.010 without another measurement. Record the two claims separately and do not interpolate across a quality curve without stating the interpolation rule.

Original worked check

A method reported faster at equal image error is not necessarily faster at equal sample count. Reproducing the wrong comparison target tests a different claim.

Practical assignment

Analyze one paper associated with R23, R44 or R55. Pass: a two-page claim map, a reconstructed central algorithm, at least three assumptions and a feasible reproduction target are reviewed by another reader.

Full execution steps and acceptance checks are in LAB_WORKBOOK.pdf under this same lesson ID. Your warm-up file is `code/exercises/lesson_41_1.py`.

Reference starting point: templates/CLAIM_EVIDENCE_MAP.md; code/run_experiments.py

Assigned study

Original assignment: R23: Further Reading sections for a chosen rendering topic; R44 or R55: one named original paper; handbook research-reading template.

Focused expansion: R44: abstract, algorithm and evaluation of the 2020 ReSTIR paper; or R55 / S05: XPBD (2016), Sections 3-5. Read the chosen paper in full before finalizing the reproduction boundary. Use S14 for artifact-review vocabulary.

Check your understanding

Attempt all three original questions before opening the key. A lesson also requires its reading, lab and graduate problems.

1. Why distinguish equal-error from equal-sample comparisons?
2. What makes a claim-evidence table useful?
3. Does a failed reproduction immediately disprove a paper?

Lesson 41.2 — Conditioning, sparse systems and iterative solvers

Learning outcome: Choose a solver from mathematical properties and error requirements.

Planning: 28-40 hours. Includes assigned study and practical work; module/Gate I time overlaps.

Original teaching primer

A linear system Ax=b has numerical behavior governed partly by conditioning: sensitivity of the solution to changes in data. Residual r=b-Ax measures equation mismatch, but a small residual need not imply a small forward error for an ill-conditioned system. Scaling and problem formulation matter before choosing a faster kernel. Sparse storage exploits structure, while iterative methods use repeated matrix-vector operations. Conjugate gradient requires a symmetric positive-definite setting for its standard guarantees. Preconditioning changes the system seen by the iteration to improve convergence without changing the intended solution when correctly applied. State stopping criteria, norm, precision and maximum work. Test a known solution and increasingly difficult conditioning, not only one friendly matrix.

Residual, forward error and the condition of the question

Let x* solve Ax=*b and let x be your estimate. Define e=x*-x and r=b-Ax. Then Ae=r, so e=A^-1 r when A is invertible. Using a consistent induced norm, ||e|| <= ||A^-1|| ||r||. Combining ||b|| <= ||A|| ||x|| *yields the relative bound* ||*e*||/||*x*|| <= kappa(A) ||r||/||b|| for nonzero b. A residual threshold alone cannot give a useful forward-error guarantee when the condition number is large.

For A=diag(1,10^-6), b=(1,10^-6) and estimate x=(1,0), the residual norm is 10^-6, while the error norm is 1. A small equation mismatch hides a completely wrong second coordinate. The example separates a well-implemented solver from a well-conditioned model. Scaling coordinates, removing redundant constraints and improving boundary conditions may matter more than faster arithmetic.

From a quadratic to preconditioned conjugate gradient

For symmetric positive-definite A, minimize E(x)=0.5 x^T A x-b^T x. Its gradient is Ax-b, and positive curvature gives a unique minimizer. CG constructs search directions that are conjugate under A rather than merely orthogonal in Euclidean space. Standard PCG additionally assumes a fixed SPD preconditioner M. Apply z=M^-1 r; do not explicitly invert a large matrix. A positive diagonal gives the simple Jacobi choice. [S01]

The teaching implementation begins with x=0, r=b, z=M^-1 r and p=z. At each step set alpha=(r^T z)/(p^T A p), update x <- x+alpha p and r <- r-alpha A p, compute z_new=M^-1 r_new, and use beta=(r_new^T z_new)/(r_old^T z_old) to form p_new=z_new+beta p. A nonpositive denominator is a breakdown signal, not a complete SPD test. A nonsymmetric input may slip past that check and still invalidate the derivation.

Storage and stopping are part of the algorithm

Compressed sparse row storage holds row offsets, column indices and nonzero values. Validate offsets, dimensions and index ranges before multiplication. For a tridiagonal problem, sparse multiplication performs work proportional to nonzeros rather than the square of the dimension. The dense solver in this package is an independent oracle for small problems, not a recommended production sparse factorization.

Stop when the measured norm satisfies max(atol,rtol*||b||), or a declared work cap is reached. Recompute the true residual before reporting success because recursive residual updates accumulate error. Report convergence status separately from the solution vector. Your experiments should include a zero right-hand side, a known solution, a poorly scaled SPD family, and an intentionally unsupported matrix. A preconditioner that costs more than it saves in iterations is not automatically an improvement.

Worked engineering example

Take A=[[4,1],[1,3]], b=(6,7), x0=0. Then r0=p0=(6,7), A p0=(31,27), r0^T r0=85 and p0^T A p0=375. The first alpha is 85/375=17/75. Thus x1=(1.36,1.5866667), which is not yet the exact solution (1,2). Verify A(1,2)=(6,7). In exact arithmetic CG needs at most two steps here; in floating point the declared tolerance determines whether the measured result is accepted. For the diagonal family in the runner, exact Jacobi scaling makes the preconditioned system the identity, so that special case can converge in one step. It does not establish that Jacobi works well on general sparse systems.

Original worked check

If A=diag(1,10^-6), its 2-norm condition number is 10^6. A small perturbation in the second equation can produce a much larger relative change in its associated solution component.

Practical assignment

Implement a sparse matrix-vector product and conjugate gradient for a verified SPD system. Pass: derive the relevant assumptions, compare residual and forward error, and evaluate a simple preconditioner over several conditioning levels.

Full execution steps and acceptance checks are in LAB_WORKBOOK.pdf under this same lesson ID. Your warm-up file is `code/exercises/lesson_41_2.py`.

Reference starting point: code/gradlab/numerical.py; code/tests/test_numerical.py

Assigned study

Original assignment: R54: linear systems, positive definiteness and numerical methods; R07: orthogonality and eigenvalues; R42: numerical optimization background.

Focused expansion: R54: positive-definite equilibrium systems; S01: Shewchuk, Sections 3, 8, 11-12 and Appendix B3; R42: numerical linear algebra background. Re-derive the two-dimensional update without copying implementation code.

Check your understanding

Attempt all three original questions before opening the key. A lesson also requires its reading, lab and graduate problems.

1. What is the 2-norm condition number of diag(1,10^-6)?
2. Why can a small residual coexist with substantial solution error?
3. What matrix property is required by standard conjugate gradient?

Lesson 41.3 — Constrained optimization, derivatives and optimality conditions

Learning outcome: Derive and verify an optimization method for an engineering objective.

Planning: 26-38 hours. Includes assigned study and practical work; module/Gate I time overlaps.

Original teaching primer

Optimization requires a defined objective, feasible set and numerical tolerance. For differentiable unconstrained problems, stationary points have zero gradient but may be minima, maxima or saddles. Convexity provides stronger global conclusions under the appropriate assumptions. Equality and inequality constraints introduce Lagrange multipliers and Karush-Kuhn-Tucker conditions, whose necessity or sufficiency depends on regularity and convexity. For nonlinear least squares, residual Jacobians support Gauss-Newton-style approximations; poor scaling or rank deficiency can require damping or regularization. Check analytic or automatic derivatives against finite differences over sensible step sizes. Gradient checks themselves have truncation and roundoff error. Report convergence histories and failed initializations rather than only a final successful parameter set.

State the optimization problem before choosing an optimizer

A fitting problem needs an objective, a feasible set and a parameterization. A low loss can coexist with impossible joint angles, a negative material coefficient or an invalid energy model. For f(x) with inequalities g_i(x)<=0 and equalities h_j(x)=0, write L=f+sum(lambda_i g_i)+sum(nu_j h_j). KKT combines primal feasibility, lambda_i>=0, stationarity of L and lambda_i g_i=0. Necessity needs appropriate regularity conditions; in convex problems, a feasible KKT point provides a global optimum under the standard convex formulation. Do not promote every stationary point to a minimum. [S02]

For an equality constraint, the multiplier is not constrained to be positive. Its sign depends on the chosen equality convention. Changing x+y-1 to 1-x-y reverses the multiplier sign without changing the solution. Numerical checks should compare stationarity and feasibility, not only a memorized multiplier.

Derive a solver you can independently verify

Minimize 0.5||x-t||^2 subject to sum(x)=s. Stationarity gives x_i-t_i+lambda=0. Summing yields lambda=(sum(t)-s)/n, so x=t-lambda*1. This affine projection is the executable reference; it is not a general nonlinear constrained solver. With t=(0.2,0.9), s=1, lambda=0.05 and x=(0.15,0.85).

For nonlinear least squares 0.5||r(theta)||^2, the gradient is J^T r and the exact Hessian is J^T J+sum(r_i Hessian(r_i)). Gauss-Newton neglects the second term; damping solves a modified system such as (J^T J+mu I)delta=-J^T r. Poorly scaled parameters, rank deficiency and large residuals can make a naive step ineffective. A line search or trust region evaluates whether the step actually improves the intended objective.

Differentiate a game-relevant model, not just a toy polynomial

For a planar two-link arm, p(theta)=(l1 cos(t1)+l2 cos(t1+t2), l1 sin(t1)+l2 sin(t1+t2)). Its first Jacobian column differentiates both links, while the second differentiates only the second link. A straight arm can lose local ability to move in one Cartesian direction: the Jacobian becomes poorly conditioned. This links inverse kinematics to Lesson 41.2 instead of treating optimizer failure as magic.

Check each derivative using centered differences at several step sizes, preferably away from discontinuities. A tolerance should reflect scale: compare absolute error near zero and relative error when the derivative is substantial. Then deliberately test a singular pose, an unreachable target and a bound-active solution. Preserve the failure report. An optimizer that returns a status and feasible fallback is more useful than one that silently returns its last iterate.

Worked engineering example

For the manifest problem f=x^2+y^2, h=x+y-1, stationarity is 2x+lambda=0 and 2y+lambda=0. Subtracting gives x=y; feasibility gives x=y=0.5 and lambda=-1. The objective is 0.5. Along the feasible line y=1-x, f=2(x-0.5)^2+0.5, which independently proves the unique minimizer. For a centered derivative check of x^3 at x=2, [(2+h)^3-(2-h)^3]/(2h)=12+h^2 in exact arithmetic. This shows the truncation term explicitly; floating-point cancellation adds a different error as h becomes very small.

Original worked check

Minimize x^2+y^2 subject to x+y=1. The Lagrangian gives 2x+lambda=0 and 2y+lambda=0; together with the constraint, x=y=0.5 and lambda=-1.

Practical assignment

Derive the example, then solve a constrained fitting or IK problem with a numerical method. Pass: derivative checks, feasibility residual, convergence criteria and at least one failure or sensitivity case are reported.

Full execution steps and acceptance checks are in LAB_WORKBOOK.pdf under this same lesson ID. Your warm-up file is `code/exercises/lesson_41_3.py`.

Reference starting point: code/gradlab/numerical.py: constrained_fit, finite_gradient

Assigned study

Original assignment: R42: convexity, duality, optimality and numerical methods; R41: gradients and constrained differentials.

Focused expansion: R42 / S02: Convex Optimization, Sections 4.1, 5.5 and 9.1-9.5; R41: gradients and constrained differentials. Study constraint qualifications separately from convex sufficiency.

Check your understanding

Attempt all three original questions before opening the key. A lesson also requires its reading, lab and graduate problems.

1. What is the constrained minimizer in the example?
2. Does zero gradient prove a general nonconvex point is a minimum?
3. Why vary finite-difference step size in a derivative check?

Lesson 41.4 — Experimental design, uncertainty and reproducible graduate work

Learning outcome: Make a performance or quality claim that survives independent scrutiny.

Planning: 22-32 hours. Includes assigned study and practical work; module/Gate I time overlaps.

Original teaching primer

Define the primary outcome and comparison before examining final results. Choose workloads that represent the intended claim, plus adverse and held-out cases. Repeated frames within one run can be correlated, so the statistical unit might be a run, scene or device rather than every frame. A large count of correlated observations does not automatically provide strong independent evidence. Use confidence intervals or other justified uncertainty summaries, report effect sizes and preserve raw observations. Ablations remove or replace components to test which parts cause an improvement. Multiple comparisons and parameter tuning can overfit the benchmark suite. Keep final evaluation data separate from tuning when feasible. State external validity limits, including devices, workloads and quality settings not tested.

Define what an observation represents

The unit of inference is the object your claim generalizes over. Frames nested in one run share scene content, thermal history and scheduling behavior. Treating every frame as an independent experiment artificially increases apparent evidence. A study about scenes may need a scene-level summary; a study about repeated execution on one machine may use independent process runs. State the hierarchy explicitly: device, scene, run and frame. [R09]

Pair a baseline and candidate on the same workload when this removes irrelevant variation. Randomize execution order to reduce systematic warmup or drift effects. Keep the seed policy fixed and explain common-random-number pairing when it is used. A run killed by an unrelated system failure may be excluded under a predefined rule; a slow run caused by the method itself is a result, not an inconvenience.

Effect size, uncertainty and diagnostic evidence

For positive paired times a_i and b_i, log speedups z_i=log(a_i/b_i) have a useful additive scale. Report exp(mean(z_i)) when a geometric-mean speedup is the chosen estimand. It is not the same as sum(a_i)/sum(b_i), which answers a total-time question. The included bootstrap resamples whole pairs and returns percentile bounds. Its numerical output cannot establish independence, repair a biased workload suite or justify a sample size.

The bundled native benchmark is a within-process diagnostic, with 12 randomized-order pairs. Those rows demonstrate data capture and timer boundaries; they are not enough to claim broad CPU or GPU superiority. For an actual study, repeat independent processes, retain environment records and justify how uncertainty is calculated at the intended unit. State the confidence procedure rather than treating a 95% interval as a 95% probability statement about a fixed unknown parameter.

Ablations and sensitivity answer different questions

An ablation removes or replaces a component while preserving other controls. A sensitivity sweep changes a parameter to reveal the boundary of an effect. Both can be misleading when quality changes simultaneously with time. Compare a denoiser with and without history at equal total budget, and also disclose the resulting error; do not call a quality loss a pure optimization.

Reserve final scenes or workload families before tuning. Choose the main metric, analysis, exclusions and stopping rule before examining the final comparison. Keep secondary findings labeled exploratory. Publish raw rows, not only a plotted mean, and give the reviewer a command that regenerates the table from those rows. Failure to reproduce a timing exactly is not automatically a failure to reproduce the engineering conclusion; define acceptable numerical and statistical agreement in advance.

Worked engineering example

Two paired workloads take baseline times (10,100) ms and candidate times (5,90) ms. The geometric-mean speedup is sqrt(2*(100/90)), approximately 1.491x. The total-time speedup is 110/95, approximately 1.158x. Both calculations are correct but weight the workloads differently. A presentation that switches between them after seeing the data changes the question. If each value was itself the mean of 1000 correlated frames, those 4000 frames do not automatically provide 4000 independent speedup observations.

Original worked check

Ten thousand frames from one scene are not equivalent to ten thousand independent games. A scene-level comparison across diverse scenes can better test a claim about broad engine behavior.

Practical assignment

Preregister a small experiment with baseline, primary metric, trial unit and exclusion policy. Pass: raw data and analysis scripts reproduce the tables, and an ablation isolates at least one proposed mechanism.

Full execution steps and acceptance checks are in LAB_WORKBOOK.pdf under this same lesson ID. Your warm-up file is `code/exercises/lesson_41_4.py`.

Reference starting point: code/gradlab/numerical.py: paired_bootstrap; evidence/reference_run/native_scan_pairs.csv

Assigned study

Original assignment: R09: statistical inference; R42: experimental examples; handbook benchmark and research templates.

Focused expansion: R09: inference, confidence intervals and experimental units; S14: artifact evaluation. Use the package PREREGISTRATION and BENCHMARK_MANIFEST templates before collecting final data.

Check your understanding

Attempt all three original questions before opening the key. A lesson also requires its reading, lab and graduate problems.

1. Why identify the statistical unit?
2. What does an ablation test?
3. Why reserve held-out workloads?

Module 41 project: numerical research notebook

Produce a notebook or script-driven report linking one research claim to a derivation, an independent small oracle and a preregistered evaluation. Implement CSR matrix-vector multiplication and PCG for a verified SPD family. Include at least one non-diagonal family; a diagonal Jacobi example alone is not enough. Compare forward error, true residual, work, setup and solve cost. Add a constrained fitting/IK problem with derivative checks, feasibility and a failure case.

Required artifacts: claim map; equations and units; generated test matrices with known solutions; derivative-step sweep; raw solver histories; preregistration; analysis script; a short reviewer response. A reasonable staged plan is mathematical contract, independent oracle, implementation, then experiments. Correctness 35 points, derivation 25, experimental design 25, reproducibility/limitations 15. Suggested pass: 80/100 with no unexplained numerical mismatch or missing assumption.

Integration checkpoint: explain how solver conditioning or research comparison choices will affect your proposed Gate I track. Do not infer a broad performance conclusion from the included eight-row diagonal diagnostic.

Module 42 — Advanced parallel and GPU algorithms

Prerequisites: Modules 18, 37-38 and 41.

Original project brief: Implement a reusable GPU reduction, scan or compaction primitive and integrate it into an engine workload. Compare a simple reference, a competent library baseline when available and your implementation at equal correctness requirements.

Core reference IDs: R37, R43, R48. Full entries are in the reading directory.

- 42.1 — Parallel reductions, scans and work-span analysis
- 42.2 — Tiling, data reuse and asynchronous pipelines
- 42.3 — Irregular workloads, wavefront scheduling and load balance
- 42.4 — Portable performance and integration-quality evaluation

Lesson 42.1 — Parallel reductions, scans and work-span analysis

Learning outcome: Prove the structure of a parallel primitive before tuning it.

Planning: 26-40 hours. Includes assigned study and practical work; module/Gate I time overlaps.

Original teaching primer

A reduction combines many values with an operator; a scan produces prefix combinations. Associativity supports regrouping mathematically, but floating-point addition is not exactly associative. Decide whether the contract requires bitwise reproducibility, bounded error or an exact integer result. A tree reduction can reduce dependency depth from linear to logarithmic while retaining linear total work. Work is the total number of operations; span is the longest dependency chain. More processors cannot beat the span without changing the algorithm. A naive parallel scan can have extra logarithmic work, while work-efficient constructions achieve linear work with logarithmic span under their model. Handle partial groups, empty inputs and irregular sizes explicitly. Cross-group coordination must follow the API rather than assuming a global barrier inside an ordinary dispatch.

Work, span and the algebraic contract

Represent the computation as a directed acyclic graph. Work W counts all operations; span S is the longest dependency path. On P ideal processors the runtime cannot be below max(W/P,S). This is a lower bound in a declared model, not a hardware prediction: memory bandwidth, launch overhead, occupancy and barriers add constraints.

A reduction needs an identity and an associative operator for arbitrary regrouping. Integer addition modulo 2^32 is associative; real addition is associative; finite-precision floating-point addition is not exactly associative. Choose the numerical contract before changing order. Determinism on one fixed tree is weaker than agreement with every serial sum or every device. The Python scan uses arbitrary-precision integers, whereas the native and CUDA examples use uint32 wraparound. Their domains agree only when you account for that difference.

Derive exclusive scan through two tree passes

For inputs (3,1,7,0,4), an exclusive sum scan is (0,3,4,11,11). The value at position i is the reduction over positions strictly before i. In an upsweep, each tree node stores the sum of its children. Set the root to the identity, then walk downward: the left child receives the parent's prefix; the right child receives the parent's prefix plus the saved left-subtree sum. This preserves the left-to-right order even for associative operators that are not commutative. [S03]

Padding a non-power-of-two input with identities simplifies a teaching implementation. If the padded size is m, the two passes use 2(m-1) combines and logarithmic depth. Since n<=m<2n for n>0, work remains O(n). Do not apply log2(0); the empty scan returns an empty sequence. The exercise should establish those cases before adding dispatch-level hierarchy.

From scan to engine compaction

A stable compaction converts visibility flags to destination offsets using exclusive scan. Write each kept element once at its offset. The output count is the last offset plus the last flag for a nonempty input. Stability matters when later processing assumes original draw order or deterministic entity ordering.

A GPU hierarchy scans chunks, scans their totals in another dispatch and adds each chunk's offset. A normal thread-block barrier does not synchronize arbitrary blocks. Do not implement a global spin barrier that assumes all blocks can be resident. The optional CUDA reference instead performs multi-dispatch reduction, with out-of-range lanes supplying zero and every lane reaching each block barrier. It is supplied for hardware validation and was not compiled in this environment. [S04]

Worked engineering example

For n=1024, a complete binary reduction performs 1023 additions with 10 combine levels. An exclusive scan built from upsweep and downsweep on 1024 leaves uses 2046 additions in this simple model. With n=1025 padded to m=2048, it uses 4094 additions, still linear in n but almost twice the previous work for one extra input. That discontinuity is a reason to benchmark boundary sizes rather than only powers of two. For flags (1,0,1,1), offsets are (0,1,1,2); kept entries go to output slots 0,1,2.

Original worked check

A balanced reduction over 1,024 elements has 10 combination levels under a binary-tree model and 1,023 total pairwise combinations. Its implementation still pays memory and synchronization costs.

Practical assignment

Implement reduction and either scan or compaction. Pass: derive work/span bounds, test sizes 0, 1 and non-powers of two, and report the numerical/reproducibility contract.

Full execution steps and acceptance checks are in LAB_WORKBOOK.pdf under this same lesson ID. Your warm-up file is `code/exercises/lesson_42_1.py`.

Reference starting point: code/gradlab/parallel.py; code/native/main.cpp; code/gpu/reduction.cu

Assigned study

Original assignment: R37: cooperative execution and synchronization; R48: parallel architectures; derive algorithm bounds using Module 14 methods.

Focused expansion: S03: GPU Gems 3, Chapter 39, work-efficient scan and large arrays; R37 / S04: CUDA SIMT programming and synchronization. Treat the historical chapter as algorithmic study, not current device tuning guidance.

Check your understanding

Attempt all three original questions before opening the key. A lesson also requires its reading, lab and graduate problems.

1. How many levels are in a balanced binary reduction of 1,024 values?
2. How many pairwise combines does that reduction require?
3. Why can parallel floating-point sums differ from serial sums?

Lesson 42.2 — Tiling, data reuse and asynchronous pipelines

Learning outcome: Overlap movement and computation without creating races.

Planning: 28-42 hours. Includes assigned study and practical work; module/Gate I time overlaps.

Original teaching primer

Tiling partitions a computation so values can be reused in a faster memory level. Matrix multiplication, image filters and neighborhood operations benefit when the tile fits available storage and exposes sufficient parallelism. Larger tiles can increase reuse but also raise register or shared-memory demand and reduce residency. Double buffering can overlap preparation of one tile with computation on another. The pipeline needs explicit ownership: no consumer reads an incomplete tile and no producer overwrites data still in use. Asynchronous copy features vary by device; maintain a correct portable baseline. Account for boundary tiles, alignment, synchronization and extra staging work. Compare arithmetic intensity at the memory level actually being optimized.

Count reuse at the memory boundary you intend to optimize

A square B-by-B matrix-multiplication tile performs about 2B^3 arithmetic operations while loading two B-by-B input tiles for one inner-product stage. With 4-byte elements, that idealized stage moves 8B^2 input bytes, giving B/4 operations per input byte. This count deliberately excludes output traffic, boundary waste, staging and repeated loads at other memory levels. Never compare that optimistic intensity with a complete measured kernel as though the boundaries were identical.

For B=32, the two input tiles need 8192 bytes; double buffering them needs 16384 bytes before padding or additional metadata. Larger B increases reuse in the model but consumes more shared storage and registers. A tile that reduces active resident work or spills registers can lose even when the operation-to-input-byte ratio improves. Use measured bottlenecks to explain a result rather than merely saying "shared memory is faster." [R43, S04]

A pipeline is an ownership protocol

Name buffer states: free, filling, ready and consuming. The producer may reuse a slot only after the previous consumer finishes. The consumer may begin only after the copy's completion event, not merely after issue. Double buffering alternates slots, but alternating indices alone does not establish either dependency.

The reference schedule has one copy engine, one compute engine and two slots. For tile i, copy begins after both the copy engine and that slot are free. Compute begins after the tile is ready and the compute engine is free. This creates an executable timing/ownership model, not actual asynchronous hardware execution. A device implementation must translate every model edge into the API's synchronization contract and handle the first and last stage explicitly.

Prove correctness before measuring overlap

Boundary tiles need predicates for each load and store. Every thread participating in a block barrier must follow a compatible control path. Zero-fill out-of-range contributions rather than letting a few lanes return before a collective barrier. Include rectangular matrices such as 5x3 multiplied by 3x7 so that every dimension has a partial tile.

For a performance comparison, separate allocation, host preparation, transfer, kernel execution and result consumption. Then measure the end-to-end path directly: overlapping stage times cannot

simply be summed. A faster steady-state pipeline can still lose for one tile because fill/drain overhead dominates. Keep a non-overlapped implementation for correctness and unsupported hardware, and log why capability dispatch selected each path. [S04]

Worked engineering example

Assume a copy takes 2 time units and computation takes 5 per tile, with independent engines and two available slots. Eight serial tiles take 8*(2+5)=56 units. The ideal pipeline takes 2+8*5=42 units: copy tile 0, then let later copies overlap computation. The model speedup is 56/42=1.333x. This is a prediction under stated assumptions, not a measured GPU result. If staging adds 2 units to each compute stage, the same model becomes 2+8*7=58 units and loses to the original serial schedule.

Original worked check

For C=A*B, reusing a loaded A or B tile across many multiply-adds can raise useful operations per byte. A tile that spills heavily can erase that advantage.

Practical assignment

Implement a tiled matrix multiply or image filter, then one overlap variant where supported. Pass: adversarial dimensions match the reference, synchronization is checked and the end-to-end comparison includes staging overhead.

Full execution steps and acceptance checks are in LAB_WORKBOOK.pdf under this same lesson ID. Your warm-up file is `code/exercises/lesson_42_2.py`.

Reference starting point: code/gradlab/parallel.py: tiled_gemm, pipeline_trace

Assigned study

Original assignment: R37: shared memory, synchronization and asynchronous operations where supported; R43: operational intensity.

Focused expansion: R37 / S04: shared memory and asynchronous execution; R43: original Roofline model. Read the chosen device API for stage completion and buffer-reuse semantics before attempting an asynchronous implementation.

Check your understanding

Attempt all three original questions before opening the key. A lesson also requires its reading, lab and graduate problems.

1. Why can larger tiles hurt performance?
2. What ownership rule is essential for double buffering?
3. Why retain a non-asynchronous baseline?

Lesson 42.3 — Irregular workloads, wavefront scheduling and load balance

Learning outcome: Design around variable work rather than assuming identical threads.

Planning: 26-40 hours. Includes assigned study and practical work; module/Gate I time overlaps.

Original teaching primer

Ray traversal, graph processing and variable-length simulation tasks have irregular work. A fixed assignment can leave some workers idle while a few long tasks dominate the tail. Work queues, compaction and wavefront processing reorganize active tasks, trading scheduling and memory traffic for better coherence or load balance. Persistent-worker designs keep workers alive to fetch tasks, but need bounded queues, termination detection and a memory-ordering contract. Global progress cannot depend on every workgroup being resident when the hardware does not guarantee that. Sorting tasks by similarity can improve coherence but has a cost and can alter latency. Evaluate both balanced and skewed distributions, including rare expensive tasks.

Throughput can look healthy while the last job dominates

For indivisible jobs with costs c_i on P workers, completion time is at least max(sum(c_i)/P,max(c_i)). A queue can redistribute jobs, but cannot make an indivisible 100-unit job finish in 10 units. Start by identifying whether a long task can be split: a ray path can move through bounded stages; a blocking external operation may not be divisible by your scheduler.

Static assignment minimizes scheduling overhead and can preserve locality. Dynamic assignment fills idle workers but adds atomic traffic, metadata movement and termination complexity. Distribution matters: uniform costs, a heavy tail and a spatially clustered hotspot can favor different strategies even at the same mean. Report makespan, queue depth and late-job latency in addition to operations per second.

Wavefront processing changes the unit of work

A monolithic path-tracing kernel carries different rays through intersection, material evaluation and secondary-ray generation. A wavefront approach places active work in stage-specific queues. This can improve coherence at a stage while increasing intermediate state traffic and dispatch cost. Compaction from Lesson 42.1 removes inactive entries between stages. Sorting by material or region adds yet another tradeoff. [S06]

For each queue, state capacity, producer count, consumer count and payload lifetime. Reserving a slot and publishing an initialized payload are different events. A consumer must not read the payload merely because an index was incremented if publication has not established the required ordering. A bounded queue must return a safe overflow/backpressure outcome, not silently overwrite live entries.

Termination needs more than an empty snapshot

An empty work queue does not prove the computation is finished while a worker can still publish descendants. One design tracks queued plus in-flight work under an ordering protocol; completion requires both to be zero with no new source admitted. Another bounds work by explicit dispatch stages and terminates when the next active count is zero. The correct choice depends on whether jobs create jobs and what progress the API guarantees.

Persistent GPU workers are not a license to spin on global barriers. Resident groups can occupy resources while waiting for groups that cannot start. Use an API-supported cooperative execution contract, a staged launch or a proof that does not require simultaneous residency. The included scheduling function models cost allocation only; it does not validate a concurrent queue, GPU memory ordering or global progress. Those remain implementation obligations. [S04]

Worked engineering example

For costs (100,1,1,...,1) with 99 one-unit jobs and 8 workers, total work is 199. The average-work lower bound is 24.875, but the indivisible-job bound is 100, so no scheduler in this model can finish below 100. Round-robin static assignment gives the first worker 100 plus 12 short jobs, for 112. A queue that starts the long job immediately can finish the rest before it, approaching 100 without dispatch overhead. Moving the long job late in queue order or charging queue costs changes the result. This is a workload model, not a measured GPU speedup.

Original worked check

If 99 tasks take 1 unit and one takes 100 units, static one-task assignments can leave the final completion dominated by the long task. Splitting or scheduling work differently may help only when the task is divisible.

Practical assignment

Implement an irregular workload using static assignment and a queue or compaction alternative. Pass: prove or test termination, handle queue capacity safely and measure tail behavior as well as mean throughput.

Full execution steps and acceptance checks are in LAB_WORKBOOK.pdf under this same lesson ID. Your warm-up file is `code/exercises/lesson_42_3.py`.

Reference starting point: code/gradlab/parallel.py: scheduling_model; code/gradlab/rendering.py: BVH

Assigned study

Original assignment: R37: execution, atomics and synchronization; R48: multithreaded scheduling; R23: wavefront rendering as an application.

Focused expansion: R23 / S06: PBRT, Mapping Path Tracing to the GPU; S04: atomics and synchronization. Relate queue publication, consumption and completion accounting to the memory model studied in Module 18.

Check your understanding

Attempt all three original questions before opening the key. A lesson also requires its reading, lab and graduate problems.

1. Why can average task cost hide poor completion time?
2. Why avoid a global spin barrier requiring every group to be resident?
3. Is sorting tasks always beneficial?

Lesson 42.4 — Portable performance and integration-quality evaluation

Learning outcome: Separate algorithmic benefit from one device-specific tuning win.

Planning: 22-34 hours. Includes assigned study and practical work; module/Gate I time overlaps.

Original teaching primer

A performance-portable implementation preserves correctness and obtains useful efficiency across its stated target set. This does not mean identical code or identical speed everywhere. Capability dispatch can select specialized paths while keeping a tested fallback. Tuning parameters should be bounded and recorded rather than silently selected from the final test set. Compare against a competent baseline and include small, large and irregular workloads. Report speed, memory, numerical error and integration cost. A kernel that is faster only after excluding transfer or conversion work should make that boundary explicit. Reproducible artifacts include build scripts, device capability reports, raw data and failure cases. Lack of access to multiple GPUs is a limitation to disclose, not a reason to invent portability evidence.

Portability has at least three separate questions

Functional portability asks whether the implementation is valid on the supported target set. Performance portability asks whether it retains useful efficiency across that set. Artifact portability asks whether another engineer can build and run it. One source file that compiles on two devices answers none of these questions completely.

Describe the target set explicitly: API, feature level, precision, storage limits and subgroup assumptions. Specialize only behind capability checks. A tile fitting on device A may be invalid on B, even before performance is considered. Avoid using a single vendor's execution width as a language-level invariant. Keep a baseline that satisfies the same numerical contract and report the reason for every fallback. [S04, R21]

The application upper bound comes before heroic kernel work

Suppose a primitive accounts for fraction f of total baseline time and is accelerated by s, with all other work unchanged. The ideal application speedup is 1/((1-f)+f/s). This derives directly from adding the unchanged and accelerated fractions; it excludes integration overhead. For f=0.2 and s=4, the maximum under those assumptions is 1/0.85, approximately 1.176x. Any added data conversion reduces that benefit.

Roofline provides a different bound: attainable arithmetic rate is no more than min(peak arithmetic rate, measured bandwidth times arithmetic intensity) under its model. Combine the two views rather than using either as a proof of achieved speed. A kernel can be bandwidth-limited while also being unimportant to the frame's critical path. [R43]

A fair comparison is a maintained artifact

Use a competent library baseline when available and disclose its version and configuration. Separate allocation, transfers, conversion, synchronization and computation; then time the end-to-end operation as used by the engine. Report small workloads where launch overhead dominates, large workloads where bandwidth matters, and irregular workloads that expose boundaries.

An untested device is a blank evidence cell, not a predicted pass. When only one GPU is available, evaluate multiple workloads on that device and narrow the claim. A CPU model can verify indexing or algebra, but cannot certify GPU performance portability. The package's native benchmark deliberately compares two CPU algorithms and preserves losses; it is a training artifact for measuring honestly, not a sales claim about a universal winner.

Worked engineering example

A frame takes 10 ms: 2 ms is the selected primitive and 8 ms is other work. A 4x primitive produces 8+0.5=8.5 ms, giving 1.176x application speedup. Now add 0.7 ms of packing and 0.4 ms of synchronization: the frame is 9.6 ms, only 1.042x faster. A report showing only the 2-to-0.5 ms kernel chart hides almost all of the engineering conclusion. If a different scene spends just 0.2 ms in the original primitive, the same fixed integration cost can turn the candidate into a regression.

Original worked check

A tile size tuned on one GPU can reduce occupancy or exceed shared-memory limits on another. Capability checks and independent tuning prevent that choice from becoming an invalid universal default.

Practical assignment

Integrate the selected primitive into the engine and evaluate at least two workload regimes. Pass: application impact is measured, unsupported features fall back safely and any untested device claims are explicitly excluded.

Full execution steps and acceptance checks are in LAB_WORKBOOK.pdf under this same lesson ID. Your warm-up file is `code/exercises/lesson_42_4.py`.

Reference starting point: code/native/main.cpp --bench; templates/CAPABILITY_MATRIX.md

Assigned study

Original assignment: R37 and R21: capability contracts; R43: performance models; Module 41 experimental protocol.

Focused expansion: R43: Roofline model; S04: capability and launch constraints; R21: the selected graphics/compute API capability contract. Use only measured hardware limits for performance estimates.

Check your understanding

Attempt all three original questions before opening the key. A lesson also requires its reading, lab and graduate problems.

1. Does portable performance require one fixed launch configuration?
2. What should be said when only one GPU was tested?
3. Why measure integration cost?

Module 42 project: reusable GPU primitive in an engine

Implement reduction plus scan or compaction, with a declared exact-integer or floating-error contract. Derive work/span, handle empty and partial groups, and use supported inter-dispatch synchronization. Integrate the primitive into an engine workload such as visible-instance compaction, particle filtering or task queue preparation.

Compare a simple independent reference, a competent available library primitive and your implementation. Include small, large and irregular sizes, boundary alignment, dispatch/setup, transfers and application consumption. Record actual target capabilities and fallbacks. A second GPU strengthens portability evidence; without one, state that only the tested device is verified. A CPU model cannot become the missing second device.

Required artifacts: API contract, device capability record, correctness/sanitizer logs where supported, before/after engine trace, raw paired timings, error results and portability matrix. Correctness and synchronization 40 points, integration 25, performance method 20, portability/limitations 15. Suggested pass: 80/100 and no race, numerical-contract violation or fabricated device result. Device evidence pending means the hardware part is not yet passed.

Module 43 — Advanced rendering and reconstruction

Prerequisites: Modules 09-10, 21-23 and 41-42.

Original project brief: Reproduce a bounded rendering result and add one justified extension. Derive the estimator or reconstruction objective, compare equal-quality cost, include temporal failure cases and preserve source scenes and evaluation scripts.

Core reference IDs: R23, R44, R47. Full entries are in the reading directory.

- 43.1 — Monte Carlo light transport and multiple importance sampling
- 43.2 — Real-time ray tracing, dynamic acceleration and hybrid pipelines
- 43.3 — Spatiotemporal reuse, reservoirs and denoising
- 43.4 — Differentiable and neural rendering under engineering constraints

Lesson 43.1 — Monte Carlo light transport and multiple importance sampling

Learning outcome: Derive estimators with correct densities and measures.

Planning: 30-46 hours. Includes assigned study and practical work; module/Gate I time overlaps.

Original teaching primer

The rendering equation expresses outgoing radiance as emission plus an integral of incident radiance, scattering and geometry factors. Monte Carlo estimation samples an integral: for samples from density p, average f(x)/p(x) where p is positive over the relevant nonzero support. Correct units and probability measures matter; area and solid-angle densities are not interchangeable without a Jacobian. Importance sampling aims to reduce variance by matching useful structure. Multiple importance sampling combines strategies with weights that sum appropriately under the estimator. The balance heuristic uses relative weighted densities, but sample counts and support must be included correctly. Russian roulette can terminate paths without bias when surviving contributions receive the proper probability compensation. Finite-variance independent sampling gives the familiar inverse-square-root standard-error scaling; do not assume it for every estimator.

An estimator is a mathematical object, not a bag of samples

For I=integral f(x) dx and samples X drawn from density p with respect to that same measure, E[f(X)/p(X)]=I when support and integrability conditions hold. A density is not a probability at a point; its units depend on the domain. In light transport, scattering terms, cosine factors, emitted radiance and visibility combine into an integrand. Losing one factor can produce plausible images while estimating the wrong quantity.

For a sampled light surface point, solid angle and surface area satisfy d_omega=abs(cos_light)/distance^2 * d_area. Therefore p_omega=p_area*distance^2/abs(cos_light). Light-selection probability is another factor: choosing one of K lights uniformly multiplies the conditional density by 1/K. Grazing directions can make p_omega large; skipping the Jacobian is not a harmless approximation. [R23, S07]

Derive multiple importance sampling with counts intact

Let technique i contribute n_i samples from p_i. An MIS estimator is sum_i (1/n_i) sum_j w_i(X_ij) f(X_ij)/p_i(X_ij). Weights must sum to one on the covered support and vanish where the associated technique cannot sample. For the balance heuristic, w_i(x)=n_i p_i(x)/sum_k n_k p_k(x). The count terms matter when sample allocation differs. This is not the same formula as drawing one random technique and forgetting its selection probability. [S07]

Use a scalar integral as an oracle before debugging full images. For f(x)=x^2 on [0,1], I=1/3. The reference combines uniform p1=1 and p2=2x, drawing p2 samples as sqrt(U). Each strategy is weighted against both densities. Repeated seeds reveal estimation noise; one seed near 1/3 proves little by itself. Support errors and density errors should be tested deliberately.

Connect the derivation to an executable path tracer

The included CPU reference has diffuse spheres, a black environment, explicit surface-light sampling, BSDF sampling, a BVH, emission handling and balance weights. BSDF throughput for cosine-sampled Lambertian reflection simplifies to albedo. When a BSDF path reaches an emitter, the light PDF is reconstructed for the same direction so the competing contributions can be weighted consistently. [S08]

Russian roulette uses a survival probability q>0 and divides surviving throughput by q. That preserves expectation for the remaining estimator, but it does not remove the reference's separate finite-depth truncation: paths longer than six bounces are omitted. State that bias. Fixed origin offsets also depend on scene scale. Linear radiance values are saved to JSON; the PPM is a tone-mapped preview and must not be used for numerical error claims. The reference is a correctness microscope, not a real-time production renderer.

Worked engineering example

For one sample from each technique, p1=2 and p2=6 give w1=2/(2+6)=0.25. With three samples from technique 1 and one from technique 2, w1=(3*2)/(3*2+1*6)=0.5. For a Lambertian surface with albedo rho under constant incoming radiance L, cosine sampling uses p=cos(theta)/pi. The contribution [rho/pi * L * cos(theta)]/p is rho*L for every nongrazing sample. With rho=0.8 and L=2, the exact reflected radiance is 1.6. This analytically checkable case catches missing pi and cosine factors.

Original worked check

With one sample from each of two strategies and densities p1=2 and p2=6 at a point, the balance weight for strategy 1 is 2/(2+6)=0.25.

Practical assignment

Implement a path tracer with explicit light sampling and MIS. Pass: derive the estimator, verify a simple known integral, compare variance at equal cost and test emission and grazing-angle edge cases.

Full execution steps and acceptance checks are in LAB_WORKBOOK.pdf under this same lesson ID. Your warm-up file is `code/exercises/lesson_43_1.py`.

Reference starting point: code/gradlab/rendering.py; code/run_experiments.py render

Assigned study

Original assignment: R23: Monte Carlo Integration, Light Transport and sampling strategies. Work the associated mathematical exercises, not only the implementation.

Focused expansion: R23: Monte Carlo Integration and Light Transport I; S07: Improving Efficiency, especially MIS; S08: A Better Path Tracer. Work through the PDF measures explicitly before porting any code.

Check your understanding

Attempt all three original questions before opening the key. A lesson also requires its reading, lab and graduate problems.

1. What is the balance weight for strategy 1 in the example?
2. Why is p(x)>0 on relevant support important?
3. What sample-count increase ideally halves standard error for independent finite-variance samples?

Lesson 43.2 — Real-time ray tracing, dynamic acceleration and hybrid pipelines

Learning outcome: Budget traversal, building and shading as one system.

Planning: 26-40 hours. Includes assigned study and practical work; module/Gate I time overlaps.

Original teaching primer

Ray-tracing acceleration structures trade build cost, memory and query cost. Dynamic scenes may update or rebuild structures; the best policy depends on movement, geometry and traversal degradation. A refit can be cheaper initially but leave poor bounds after large changes. Hardware-assisted traversal still needs correct synchronization and resource lifetime. Hybrid renderers combine rasterized visibility with ray-traced shadows, reflections or indirect lighting. Each path needs a consistent material and coordinate model. Screen-space or cached approximations require fallbacks where information is absent. Measure acceleration build/update, tracing, shading, denoising and synchronization separately, then report total frame impact. Compare equivalent scene motion and image quality rather than static screenshots alone.

The best acceleration structure depends on its lifetime

A hierarchy groups primitives into bounds so a ray can reject many primitives at once. A tighter or more carefully partitioned tree may cost more to build yet reduce traversal. Refit preserves topology and updates bounds after motion; rebuilding changes the partition. Evaluate total build/update plus query work over the reuse interval, not just the shortest traversal time.

For a structure rebuilt once and reused for F frames, per-frame amortized cost is build/F+trace, assuming comparable frames. That assumption can fail under motion: a cheap refit can leave broad overlapping bounds that worsen traversal. A policy should consider predicted or observed query cost, memory and update frequency. The reference uses a median-split sphere BVH rather than a sophisticated surface-area-heuristic builder, giving the learner room for a meaningful extension.

Correctness includes degenerate rays and changing geometry

A slab test intersects a ray interval against each axis-aligned box. For a zero direction component, check whether the origin lies inside that slab instead of dividing by zero. Intersections must use a consistent t interval and handle rays starting inside bounds. Compare the accelerated result against brute force for randomized rays, then test tangency, parallel rays and close surfaces explicitly.

The supplied refit assumes the same primitive identities and topology remain present. Adding or removing primitives is not covered by that operation. A dynamic-scene test should move geometry enough to change overlap and query coherence, rather than only translating the whole scene together. Track primitive tests or node visits as explanatory metrics; elapsed time alone may hide why a refit loses.

Hybrid pipelines share resources and meaning

A rasterized primary view can feed ray-traced shadows or reflections, but a ray effect must still agree about materials, coordinate frames and visibility. Screen-space information cannot describe an offscreen occluder, so a fallback must state what happens when information is missing. Do not quietly compare a screen-space approximation with full-scene rays as if they had identical quality.

Hardware acceleration adds build resources, scratch memory, device addresses, synchronization and object lifetimes. Keep source geometry and scratch allocations valid for the durations required

by the API, and establish a build-write to trace-read dependency. The existence of a ray-tracing feature does not establish that every update mode or geometry change is valid. The CPU model validates intersection logic only; it does not validate those hardware lifetime requirements. [S09]

Worked engineering example

Policy A takes 1 ms to refit and 5 ms to trace each frame. Policy B takes 5 ms to rebuild and 2 ms to trace. If every frame uses a fresh structure, A costs 6 ms and B costs 7 ms, so rebuilding loses. If a static structure is built once and reused for F frames, total times are 1+5F and 5+2F. B wins when 5+2F<1+5F, meaning F>4/3, so it wins from two identical-use frames onward under that simplified model. In a moving scene, the fixed 5 ms trace assumption must be retested.

Original worked check

A faster traversal variant that adds 4 ms of build work to save 1 ms of tracing is a regression for a scene rebuilt every frame, but might help a scene reused for many frames.

Practical assignment

Add one ray-traced effect to the engine or build a software hybrid prototype. Pass: dynamic-scene tests include total update cost, missing-information fallbacks and resource synchronization evidence.

Full execution steps and acceptance checks are in LAB_WORKBOOK.pdf under this same lesson ID. Your warm-up file is `code/exercises/lesson_43_2.py`.

Reference starting point: code/gradlab/rendering.py: BVH, Sphere, path_radiance

Assigned study

Original assignment: R23: intersection acceleration and light transport; R21, R24 or R25: supported ray-tracing interfaces for an optional hardware route.

Focused expansion: R23: Primitives and Intersection Acceleration; S09: Vulkan acceleration-structure specification for an optional hardware route. Follow the exact API version used by your project for build/update flags and synchronization.

Check your understanding

Attempt all three original questions before opening the key. A lesson also requires its reading, lab and graduate problems.

1. Why can a cheap BVH refit become a poor long-term choice?
2. Is saving 1 ms of tracing worth 4 ms of per-frame rebuild cost?
3. Why compare animated scenes?

Lesson 43.3 — Spatiotemporal reuse, reservoirs and denoising

Learning outcome: Evaluate variance reduction together with bias, history validity and motion.

Planning: 30-46 hours. Includes assigned study and practical work; module/Gate I time overlaps.

Original teaching primer

Temporal and spatial reuse exploit related samples across frames or nearby locations. Reusing data changes the sampling and correlation structure, so the estimator and weights must be derived rather than improvised. Reservoir sampling can retain a compact representative sample under a defined weighted selection rule; its rendering use requires additional treatment of target distributions and reuse. Denoising reconstructs a signal from noisy observations using assumptions about spatial, temporal or learned structure. It can suppress noise while introducing bias, blur or ghosting. Visibility changes and incorrect motion vectors invalidate history. Distinguish estimator bias from reconstruction bias and assess moving sequences at equal total cost. The original 2020 ReSTIR paper is a historical reproduction option, not a claim to represent the latest technique.

Prove the weighted reservoir before reusing it

For nonnegative weights w1,...,wM, keep a running sum W and replace the current selection with item i with probability wi/W after adding its weight. Inductively, an earlier item's selection probability becomes [wj/W_old]*[W_old/W_new]=wj/W_new. The newly arrived item has probability wi/W_new. Thus the final item is selected proportionally to its weight. Reject negative and nonfinite weights. If all weights are zero, no representative is selected.

This proves a selection rule, not a rendering estimator. The reference's frequency test checks weights (1,2,7), expecting frequencies near (0.1,0.2,0.7). Statistical tolerance is declared for a fixed seed and trial count; it is not a proof of every distributional property. The all-zero case and overflow behavior are just as important as the attractive positive-weight case.

Selection weights and estimator normalization are different

Consider M independent candidates Xi~q, target surrogate p_hat positive wherever the desired contribution f is nonzero, and weights wi=p_hat(Xi)/q(Xi). After weighted selection of Y, the estimator [W/(M*p_hat(Y))]*f(Y) has conditional expectation (1/M) sum_i f(Xi)/q(Xi). Taking expectation over candidates gives the desired integral when support and integrability conditions hold. This derivation explains why keeping only Y without W and M loses essential information.

Spatial or temporal reservoir reuse changes the candidate-generation structure, the target and the correlations. Reusing a neighbor's representative as though it were a fresh draw from your proposal does not inherit the derivation above. The selected research method supplies additional normalization and validity conditions; derive those for the exact bounded variant being reproduced. Do not call a weighted reservoir implementation "ReSTIR" by itself. [R44]

History acceptance is a geometric and statistical decision

A temporal filter blends current c with reprojected history h: output=alpha*c+(1-alpha)*h for valid correspondence, otherwise output=c. The same pixel address is not correspondence when objects or the camera move. Depth, surface identity, normals, motion vectors and visibility changes can provide rejection evidence, but thresholds themselves create tradeoffs.

Noise reduction can introduce blur, lag, ghosting or estimator/reconstruction bias. Test disocclusion, a camera cut, moving highlights and changing illumination. Separate raw estimator error from reconstruction error by evaluating both before and after filtering. Sequence-level metrics and selected temporal traces expose failures that a favorable still frame hides. Total cost includes history storage, motion processing, rejection and reconstruction, not only rays saved.

Worked engineering example

Suppose an old pixel is 1 and a newly revealed background is 0. With alpha=0.2 and no rejection, the filter outputs 0.8, then 0.64, then 0.512 on successive zero observations. After ten frames the remnant is 0.8^10, approximately 0.1074. The noise-free input has acquired a visible error entirely from invalid history. Correct rejection outputs 0 immediately in this simplified case. A denoiser can therefore lower static-frame noise while becoming less accurate at a visibility transition.

Original worked check

If a moving object reveals a background, old foreground samples are not valid background evidence merely because they occupy the same pixel coordinate. History acceptance must account for correspondence and visibility.

Practical assignment

Reproduce a small weighted-reservoir example or temporal denoiser, then evaluate on motion and disocclusion sequences. Pass: the selection/weight rule is derived, bias limitations are stated and results include sequence-level failure cases.

Full execution steps and acceptance checks are in LAB_WORKBOOK.pdf under this same lesson ID. Your warm-up file is `code/exercises/lesson_43_3.py`.

Reference starting point: code/gradlab/rendering.py: Reservoir, temporal_filter

Assigned study

Original assignment: R44: original ReSTIR paper and its estimator assumptions; R23: sampling theory; read only the selected bounded method before expanding scope.

Focused expansion: R44: Bitterli et al., ReSTIR (2020), estimator and reuse assumptions; R23: sampling theory. The executable Reservoir class is only weighted stream selection, not the complete paper algorithm.

Check your understanding

Attempt all three original questions before opening the key. A lesson also requires its reading, lab and graduate problems.

1. Why cannot reused samples always be treated as new independent samples?
2. Why is same-pixel history not enough to establish correspondence?
3. What should be compared in a denoised rendering benchmark?

Lesson 43.4 — Differentiable and neural rendering under engineering constraints

Learning outcome: Test learned or gradient-based methods against explicit baselines.

Planning: 28-44 hours. Includes assigned study and practical work; module/Gate I time overlaps.

Original teaching primer

Differentiable rendering studies how outputs change with scene or model parameters. Visibility discontinuities, stochastic sampling and approximations complicate gradients. A useful derivative must match the actual objective and parameterization; automatic differentiation alone does not guarantee the right physical or statistical estimator. Validate on small scenes with finite differences and known cases. Neural rendering or learned reconstruction can trade preprocessing, training and memory for runtime quality or speed. Training data coverage, scene specificity, temporal stability and inference hardware constrain deployment. Compare a non-learned baseline and include training or preprocessing cost where the claim requires it. Separate interpolation within familiar conditions from generalization to new scenes, lighting or motion.

Differentiate the actual inverse problem

Let observations y_i be generated by a rendering model R_i(theta). A squared-error objective is 0.5 sum_i (R_i(theta)-y_i)^2, with gradient sum_i (R_i-y_i)*dR_i/dtheta. Differentiating an implementation produces derivatives of that implementation's computational path. It does not automatically account for omitted visibility changes, discontinuous sampling choices or a mismatch between the modeled camera and the data. [S10]

Start with a smooth, identifiable subproblem: known lighting and scalar Lambertian albedo. Write R_i=rho*b_i with b_i=E*max(cos_i,0)/pi. The unconstrained optimum is rho=sum(b_i*y_i)/sum(b_i^2), projected to [0,1] for the physical albedo bound. The denominator measures sensitivity; all-dark observations provide no information about rho. The executable reference rejects that unidentifiable case instead of inventing an estimate.

Gradient checks need a noise and discontinuity policy

For deterministic smooth models, compare analytic derivatives with centered differences over a range of step sizes. For stochastic estimators, independent random samples at theta+h and theta-h can make the difference dominated by noise. Common random numbers can reduce variance when used correctly, but do not make a discontinuous or biased derivative estimator mathematically valid.

A silhouette crossing can change which surface is visible abruptly. A local derivative computed while holding the sampled path fixed may miss a boundary contribution to the desired derivative. State whether the experiment excludes visibility changes, uses an approximation or employs a method specifically designed for them. The small albedo fit deliberately keeps visibility and illumination fixed; it is a baseline, not a full differentiable renderer.

A learned method carries an additional engineering bill

For a neural route, define training inputs, targets, architecture, loss, optimizer, random seeds and stopping criterion. Hold out entire scenes when correlated frames would leak content. Compare with a non-learned baseline such as a simple temporal/spatial filter or analytic fit, not only with a deliberately undertrained network. Report training/preprocessing cost, inference time, model weights, activation memory and temporal behavior. [R47]

A preparation cost C amortized over N uses contributes C/N per use. Training can be acceptable for a repeatedly reused level yet impractical for user-generated content changed every minute. Distribution shifts include new materials, lighting, camera motion or hardware precision. Keep the performance claim limited to the tested shifts, and preserve a fallback that has a known numerical and latency contract.

Worked engineering example

Let E=1, cosine values (0.2,0.5,1) and true rho=0.7. Then b=cosine/pi and y=0.7*b. The numerator of the least-squares formula is 0.7 sum(b^2), so the fitted albedo is exactly 0.7 in exact arithmetic. If you instead try to estimate both rho and unknown E from these data, only their product is observable: doubling E and halving rho leaves predictions unchanged while both remain valid. More optimizer iterations cannot remove that identifiability ambiguity; it needs a known light, a prior or additional observations.

Original worked check

A reconstruction network tested on frames from its own training sequence may perform well without generalizing to a new level. Holding out entire scenes tests a stronger claim.

Practical assignment

Implement a small inverse-rendering optimization or learned reconstruction experiment. Pass: gradient or generalization checks, a non-learned baseline, compute/memory accounting and a distribution-shift failure analysis are included.

Full execution steps and acceptance checks are in LAB_WORKBOOK.pdf under this same lesson ID. Your warm-up file is `code/exercises/lesson_43_4.py`.

Reference starting point: code/gradlab/rendering.py: inverse_albedo; code/gradlab/numerical.py: finite_gradient

Assigned study

Original assignment: R47: automatic differentiation, optimization and generalization; R23: light transport and sampling foundations; select one original method with Module 41's paper-review protocol.

Focused expansion: R47: automatic differentiation and generalization; S10: Mitsuba gradient-based optimization tutorial; R23: the light-transport model you differentiate. Choose an original neural method only for the optional learned route.

Check your understanding

Attempt all three original questions before opening the key. A lesson also requires its reading, lab and graduate problems.

1. Does automatic differentiation guarantee the desired gradient estimator?
2. Why hold out entire scenes for some rendering evaluations?
3. When should training cost be included?

Module 43 project: bounded rendering reproduction

Select one result from an original rendering method and reconstruct its estimator or objective. A diffuse MIS renderer, a specific reservoir-reuse variant, a temporal reconstruction method or a smooth inverse-rendering problem can provide the setting, but a primitive alone is not a complete reproduction of a larger paper.

Preserve scene geometry, camera paths, lights, materials, seeds and linear reference images. Measure error against an adequately converged or analytically known reference, including reference uncertainty. Compare at equal total cost and at a declared quality threshold where feasible. Add motion/disocclusion or a justified adverse setting. Separate sampling error from reconstruction bias and include build, shading, history and preprocessing costs appropriate to the claim.

Required artifacts: paper-to-code map, estimator derivation, source scenes, raw linear outputs, error/time analysis, temporal failure evidence, one extension and limitations. Estimator/objective correctness 35 points, baseline/equal-quality comparison 25, temporal/adverse cases 20, artifact/review 20. Suggested pass: 80/100 and no unresolved density-measure or data-leakage error.

Module 44 — Advanced engine systems and simulation research

Prerequisites: Gates F-H; Modules 41-43.

Original project brief: Conduct an engine-system experiment with a formal model or numerical argument, a reference implementation and stress-tested integration. Choose one main depth area: world streaming, simulation, distributed state or reliability.

Core reference IDs: R20, R26, R54, R55, R50. Full entries are in the reading directory.

- 44.1 — Large-world scheduling, streaming and critical-path control
- 44.2 — Deformable simulation, variational models and solver tradeoffs
- 44.3 — Distributed game state, consistency and overload behavior
- 44.4 — Formal specifications, replay and reliability-oriented engine design

Lesson 44.1 — Large-world scheduling, streaming and critical-path control

Learning outcome: Design a bounded system under changing content and workload.

Planning: 26-40 hours. Includes assigned study and practical work; module/Gate I time overlaps.

Original teaching primer

Large-world engines coordinate visibility, simulation, streaming and resource residency under changing demand. A frame graph can make dependencies explicit, but hidden shared state or unpredictable tasks can still expand the critical path. Partition work by ownership and locality while controlling cross-partition communication. World-coordinate precision can degrade far from the origin; origin rebasing, hierarchical coordinates or higher precision each affect physics, rendering and networking interfaces. Streaming priorities should reflect gameplay deadlines and resource availability, not only distance. Build a workload model with bursts, cancellation and scarce memory. Evaluate tail latency and quality degradation under stress rather than only steady travel through a preloaded scene.

Treat streaming as admission control, not a list of loads

A request consumes more than resident asset memory. Its compressed input, decompression scratch, decoded output, upload staging and temporary old version may overlap in lifetime. Define separate budgets and the ownership of each reservation. Admission should account for the peak overlap, not only the final asset size. A bounded queue prevents unbounded metadata growth; a resident-byte limit does not by itself bound pending allocations.

Assign a deadline, priority, estimated cost and cancellation identity to each request. Distance may predict demand during ordinary travel, but a teleport defeats that prediction. A policy can retain a low-detail representation, refuse low-priority work or show a placeholder while recovering. Those are deliberate quality/availability changes. Blocking the frame until every missing asset arrives is also a policy, usually with a very different latency contract.

The frame deadline follows dependencies

Represent frame tasks as a directed acyclic graph. Its critical path is the longest dependency chain, including synchronization and required data movement. Adding workers cannot remove a serial dependency. A background task can still delay the frame through a shared allocator lock, memory pressure or a mandatory join. Instrument readiness, queue wait, service, completion and consumption separately rather than timing only the task body.

An epoch identifies a world request generation. On teleport, increment the epoch and cancel obsolete work. A late completion must check its epoch before publishing a resource; cancellation cannot assume that a device operation already in flight instantly stops. Publication and resource retirement need explicit lifetime rules. The supplied model clears queued requests on epoch changes, but implementing late asynchronous completions is a required lab extension, not an already tested feature.

Large coordinates cross subsystem boundaries

In a binary floating-point format, absolute spacing grows with the magnitude of a coordinate. A single-precision value near 2^20 has spacing 2^(20-23)=0.125 in its represented units. Millimeter-level changes cannot all be represented there. A local origin or cell-plus-offset representation separates large world location from fine local motion; merely switching one render variable to double precision does not synchronize physics, networking and saved references.

Define which coordinate space every interface accepts. During rebasing, preserve object identity and transform histories consistently, including previous-frame positions used for motion vectors. Test transitions across cell boundaries and the origin shift during outstanding streaming work. Preserve a stable world-space identity for persistence; a temporary local coordinate should not become the sole database key.

Worked engineering example

Suppose the frame needs CPU tasks A=2 ms, B=3 ms after A, and C=4 ms after A, followed by D=1 ms after both B and C. Total work is 10 ms, but the ideal critical path is A+C+D=7 ms. With two workers and no other costs, 7 ms is attainable; dividing total work by two and claiming 5 ms ignores dependencies. If D additionally waits for a 5 ms upload started only after C, the path grows to 12 ms. Moving the upload to a background queue does not remove the dependency if D still needs its result.

Original worked check

A camera teleport invalidates a distance-based prefetch assumption and can create a sudden working-set change. The engine needs placeholders and a bounded recovery policy rather than an unlimited synchronous load.

Practical assignment

Implement a bounded world-streaming scheduler or large-coordinate experiment. Pass: teleport and memory-pressure tests preserve correctness, peak resources are measured and the critical path is explained with a trace.

Full execution steps and acceptance checks are in LAB_WORKBOOK.pdf under this same lesson ID. Your warm-up file is `code/exercises/lesson_44_1.py`.

Reference starting point: code/gradlab/systems.py: StreamModel, Request

Assigned study

Original assignment: R26: world, task and streaming systems for comparison; R20: engine architecture; R54: numerical conditioning where coordinate precision is studied.

Focused expansion: R20 and R26: engine task, world and streaming architecture; R54: numerical representation. The reference is a discrete-event resource model, not asynchronous disk or GPU I/O.

Check your understanding

Attempt all three original questions before opening the key. A lesson also requires its reading, lab and graduate problems.

1. Why can a camera teleport defeat ordinary prefetching?
2. Why is origin rebasing an interface issue?
3. What distinguishes a bounded degradation policy?

Lesson 44.2 — Deformable simulation, variational models and solver tradeoffs

Learning outcome: Derive a physical or constraint model before comparing visual results.

Planning: 30-46 hours. Includes assigned study and practical work; module/Gate I time overlaps.

Original teaching primer

Deformable simulation represents degrees of freedom and a model of forces, energies or constraints. Spatial discretization, boundary conditions and time integration together define the numerical problem. Finite-element or mass-spring approaches approximate different structures; a visually similar result does not prove equivalent material behavior. Implicit methods can improve stability for stiff systems while requiring nonlinear or linear solves and introducing other errors. Position-based and compliant-constraint methods offer alternative formulations with their own parameters and convergence behavior. Derive one simple energy or constraint, explain its units and study timestep/iteration sensitivity. Separate stability, accuracy and physical fidelity. A stable method can be inaccurate, and an accurate local solver can still use an inappropriate material model.

Start from energy, mass and a time discretization

For a one-dimensional spring displacement C(x)=x-rest, energy is E=0.5*k*C^2 and force is -k*C. If k has units N/m, compliance alpha=1/k has units m/N. A multi-particle constraint has gradients with respect to every participating position and inverse masses that govern how a correction is distributed. A pinned particle has inverse mass zero; it must not move under the constraint correction.

For implicit Euler on a linear spring system with mass matrix M, stiffness K and step h, eliminating velocity gives (M+h^2*K)*x_next=M*(x+*h*v)+h^2*f, with rest offsets incorporated consistently in the force or displacement formulation. This is a numerical approximation to the dynamics, not the exact continuous trajectory. An independently assembled dense solution is a useful oracle for a small sparse/iterative implementation.

Derive compliant projection and preserve the multiplier

XPBD introduces scaled compliance alpha_tilde=alpha/h^2. For one constraint, the multiplier increment is delta_lambda=(-C-alpha_tilde*lambda)/(sum_i w_i*||grad_i C||^2+alpha_tilde). Positions receive delta_x_i=w_i*grad_i C*delta_lambda. Accumulate lambda across solver iterations within the step; resetting it at every iteration changes the method. The simple reference resets it at each new timestep and retains it across that timestep's iterations. [S05]

For a scalar unit-mass constraint with alpha=0.01, h=0.1 and predicted displacement 1, alpha_tilde=1. Starting lambda=0 gives delta_lambda=-1/(1+1)=-0.5, so the corrected displacement is 0.5. The accumulated lambda is -0.5 and the constraint-plus-compliance residual is 0.5+1*(-0.5)=0. An extra identical iteration makes no change. This is compliant equilibrium for the step, not a failed attempt to enforce C=0.

Separate model, discretization and solve error

A simulation has at least three distinct error sources: the chosen physical model, time/space discretization and incomplete numerical solution. Changing iterations primarily changes solve error. Changing h changes the discrete time problem as well as alpha_tilde. Changing mesh spacing without rescaling material parameters can change the represented physical material. Vary these independently and compare at equal simulated time.

The executable bar is a pinned one-dimensional chain of unit-mass displacements with linear neighboring springs. Its converged XPBD result can be compared with a separately assembled implicit-Euler solve of the same discrete problem. It is not a three-dimensional cloth implementation, collision solver or general nonlinear finite-element model. Use its agreement as a stepping stone to a bounded paper result; do not generalize it to all deformable simulation. Stable pictures alone do not establish accurate forces, constraint errors or material behavior.

Worked engineering example

For the scalar values above, resetting lambda to zero on each iteration produces x=0.5, then 0.25, then 0.125. The solver incorrectly drives a compliant spring toward a hard constraint as iterations increase. Keeping lambda gives x=0.5 after the first iteration and thereafter. This one-variable counterexample isolates the role of multiplier accumulation more clearly than a cloth screenshot. Use the reset-every-iteration variant as an explicitly incorrect ablation, not as an alternative implementation of XPBD.

Original worked check

For spring energy E=0.5*k*(x-rest)^2, the force is -dE/dx=-k*(x-rest). This derivative supplies a check before extending to a multi-particle system.

Practical assignment

Implement a small cloth, elastic bar or compliant-constraint model and compare two integration/solver choices. Pass: derive the core equations, check gradients, quantify constraint or energy error and reproduce one bounded result from the assigned research.

Full execution steps and acceptance checks are in LAB_WORKBOOK.pdf under this same lesson ID. Your warm-up file is `code/exercises/lesson_44_2.py`.

Reference starting point: code/gradlab/systems.py: xpbd_scalar, bar_step, simulate_bar; code/run_experiments.py: bar

Assigned study

Original assignment: R54: equilibrium, variational and differential-equation methods; R55: 2016 XPBD and related position-based simulation course notes.

Focused expansion: S05 / R55: Macklin, Mueller and Chentanez, XPBD (2016), equations 10-18; R54: variational mechanics and linear systems. Reproduce the selected result before calling an extension research evidence.

Check your understanding

Attempt all three original questions before opening the key. A lesson also requires its reading, lab and graduate problems.

1. What force follows from the example spring energy?
2. Does numerical stability guarantee physical accuracy?
3. Why vary timestep and solver iterations independently?

Lesson 44.3 — Distributed game state, consistency and overload behavior

Learning outcome: Specify which guarantees survive network delay and partial failure.

Planning: 26-40 hours. Includes assigned study and practical work; module/Gate I time overlaps.

Original teaching primer

Distributed engine services divide state across processes or machines. Some operations require a single authoritative order, while others tolerate delayed or approximate views. Consistency is a property of a specified operation and model, not a single adjective that applies equally to every subsystem. During communication failure, availability and agreement requirements can conflict. Partition ownership, migration and recovery need explicit protocols. A world handoff must not create two authorities granting the same irreversible outcome. Queues require bounds and backpressure; retry storms can magnify overload. Test fault sequences involving delayed messages, crashes and duplicate requests. Use local process simulation before public infrastructure, and never claim internet-scale behavior from a two-process happy-path demo.

Specify the operation that requires authority

A position snapshot may tolerate delay while granting a unique item requires an unambiguous committed decision. Define operations, their observable effects and the consistency requirement of each. A server believing it owns an entity is not sufficient when old messages can arrive after ownership changes. The receiver that commits an irreversible effect must enforce the ownership rule.

A fencing token is a monotonically advancing ownership epoch checked by that committing receiver. After epoch 8 replaces epoch 7, an epoch-7 request must be rejected even if its sender still believes its lease is valid. The transfer of ownership itself needs a correctly coordinated, durable protocol in a real distributed system. The teaching model assumes a single atomic store that updates authority and checks commits; it does not implement replicated consensus or survive an actual machine power loss. [S11, S17]

Make duplicate handling part of the transaction

An accepted operation needs an idempotency identity and a recorded result. If a client times out after the server commits, its retry must not repeat the effect. Check the request ID, validate its payload consistency, apply the effect and record its result atomically relative to competing operations. A repeated ID with a different payload is a conflict, not a free new request.

Deduplication storage is finite. Expiring an ID without a declared retry horizon can make a very late duplicate appear new. Keeping every ID forever is an unbounded resource promise. State a retention/epoch policy or reject new work when the bounded educational store fills. Exactly-once effects require a specified boundary and failure model; a transport protocol's delivery behavior does not by itself guarantee the application's transaction semantics.

Test histories, not just individual error codes

A useful fault schedule is: old owner sends a request, the request is delayed, ownership transfers, new owner commits, the delayed old request arrives, then a retry is duplicated. Check the complete committed history and invariant, not only whether each function returned a plausible status. Introduce crashes before and after the imagined commit boundary to expose what must be persisted.

Under overload, bound ingress queues, outstanding requests and retries. Backoff with jitter can reduce synchronization, but a retry budget and deadline are still needed to stop unlimited amplification. Distinguish rejection, timeout and unknown commit outcome. Test the recovery path when the deduplication capacity is exhausted. Local deterministic schedules are excellent semantic tests; they are not measurements of internet latency or evidence of global service reliability.

Worked engineering example

Owner A holds epoch 7. Its award request is delayed. Authority changes atomically to B at epoch 8; B commits award request R. When A's epoch-7 operation reaches the authoritative sink, the sink rejects it before applying an effect. A retry of B's same request R returns its recorded result without re-awarding. Without the sink's epoch check, a correct-looking handoff acknowledgment cannot stop A's delayed request from committing. Without atomic deduplication, two simultaneous copies of R could both pass a separate check and award twice.

Original worked check

If two servers both accept ownership of the same entity during a failed handoff, both might award a unique item. A versioned ownership or fencing mechanism must prevent stale authority from committing that effect.

Practical assignment

Model a two-partition entity handoff with injected failure. Pass: a stated safety invariant holds, recovery assumptions are explicit and overload is bounded without unbounded retry amplification.

Full execution steps and acceptance checks are in LAB_WORKBOOK.pdf under this same lesson ID. Your warm-up file is `code/exercises/lesson_44_3.py`.

Reference starting point: code/gradlab/systems.py: FencedStore

Assigned study

Original assignment: R17-R18: transport and transactions; R20: game-state architecture; use Module 41's claim and experiment discipline for the distributed model.

Focused expansion: S11: Ongaro and Ousterhout, Raft, replicated state-machine assumptions; S17: Consul session and locking semantics. These sources distinguish coordinated authority from a local locking intuition; the supplied store is not a consensus implementation.

Check your understanding

Attempt all three original questions before opening the key. A lesson also requires its reading, lab and graduate problems.

1. Why is duplicate authority dangerous?
2. What is a retry storm?
3. Can a small local demo establish internet-scale reliability?

Lesson 44.4 — Formal specifications, replay and reliability-oriented engine design

Learning outcome: Use precise invariants to reduce the space of subtle failures.

Planning: 28-44 hours. Includes assigned study and practical work; module/Gate I time overlaps.

Original teaching primer

A formal model abstracts a system to states and transitions so properties can be examined precisely. Safety states that something bad never happens; liveness states that something good eventually happens under assumptions. An engine resource manager can have safety invariants about ownership and liveness obligations about eventual completion or cancellation. Model checking explores the chosen abstraction, not every detail of the production implementation. Connect model transitions to real code and tests. Deterministic replay, fault injection and property-based testing complement formal reasoning by exercising implemented behavior. Avoid vacuous proofs from impossible environment assumptions. A reliability claim should identify the property, model, bound and implementation evidence separately.

Define states and transitions before selecting a prover

A formal specification starts with variables, allowed initial states and a transition relation. For an asset request, useful state includes owner-alive, request-issued, request-pending and whether an invalid callback occurred. State the invariant: a callback must not touch an owner after release. A transition that completes pending work must check owner validity or use a lifetime mechanism that guarantees the target still exists.

Safety says that forbidden behavior never occurs. A finite bad prefix can demonstrate a safety violation. Liveness instead demands eventual progress, such as each accepted request eventually completing or being canceled. A model that allows the worker to be ignored forever cannot promise completion without a scheduling or fairness assumption. Fairness is a substantive environment requirement, not a magic word added after the proof. [S12]

A finite model can be complete for itself and incomplete for the engine

Breadth-first search of all reachable states is exhaustive for a finite model when its transition relation and initial states are fully represented. Store predecessor edges to recover a minimal-step counterexample. The supplied one-request model explores this graph and deliberately disables the callback guard to expose the sequence request, release, complete. Passing the guarded model says nothing by itself about arbitrary request counts, memory reclamation, real pointers or device callbacks.

Connect each model transition to code and an implementation test. Check that assumptions do not eliminate the very failures you intend to study. For example, declaring release impossible while work is pending would make the ownership test vacuous if real callers can release then. Mutation testing helps: a deliberately missing guard must produce a counterexample, proving that the checked property can detect the intended class of mistake.

Replay preserves evidence only when nondeterminism is captured

A seed alone is not always a replay record. Input order, clock reads, external responses, thread scheduling, floating-point environment and random-number consumption can affect results. Define which nondeterminism is recorded, controlled or excluded. Hash canonical serialized states at checkpoints and compare the first divergence, but retain the underlying state or event log so a mismatch is diagnosable.

The reference hash sorts JSON object keys and uses a stable digest; it is not a cross-platform floating-point determinism guarantee or a cryptographic proof of correct simulation. Replay and fault injection complement model checking by exercising actual code. Keep the evidence labels separate: model safety, conditional liveness argument, implementation tests and measured reliability are different claims.

Worked engineering example

Start with alive=true and no pending request. The request transition sets pending=true. Release sets alive=false. In the buggy model, complete unconditionally dispatches the callback, setting bad=true. The resulting three-transition trace is a safety counterexample. In the guarded model, complete clears pending but does not call the dead owner. This fixes the stated safety property. It does not prove liveness: a scheduler that never executes complete or cancel can leave pending=true forever. A progress assumption and separate liveness analysis are required.

Original worked check

For an asset request, safety might require that a released owner never receives a callback touching its memory. Liveness might require that each accepted request eventually completes or cancels, assuming the worker continues to run.

Practical assignment

Model a small engine lifecycle or queue protocol and connect it to implementation tests. Pass: at least one safety and one conditional liveness property are defined, a deliberate violation is found and abstraction limits are documented.

Full execution steps and acceptance checks are in LAB_WORKBOOK.pdf under this same lesson ID. Your warm-up file is `code/exercises/lesson_44_4.py`.

Reference starting point: code/gradlab/systems.py: lifecycle_check, replay_hash

Assigned study

Original assignment: R50: safety, liveness and assumptions as a formal-methods reference; R11 and Module 18: ownership/concurrency contracts; R26: system comparison.

Focused expansion: S12: Lamport, Safety, Liveness, and Fairness; R50: formal assumptions and bounded verification. The included exhaustive search establishes safety only for its small explicit model.

Check your understanding

Attempt all three original questions before opening the key. A lesson also requires its reading, lab and graduate problems.

1. How do safety and liveness differ?
2. Why must liveness assumptions mention scheduling or worker progress?
3. Does proving an abstract model automatically prove all implementation code?

Module 44 project: one engine-system depth experiment

Choose world streaming, deformable simulation, distributed state or lifecycle reliability as the main depth area. Complete all four lesson labs at their bounded scale, then integrate one area more deeply. Write a numerical argument or state/transition model before comparing implementations.

Streaming evidence must include teleport, late completion, transient memory and a bounded degradation policy. Simulation evidence must separate model, discretization and solve error with timestep/iteration sweeps. Distributed-state evidence must identify the authoritative sink, deduplication/commit boundary and persistence assumptions under fault histories. Reliability evidence must distinguish checked safety from conditional liveness and connect the model to implemented tests.

Required artifacts: contract, reference model, implementation, deliberately failing case, stress trace, parameter sweep and limitations. Model/argument 30 points, implementation correctness 30, failure/overload evaluation 25, reproducibility 15. Suggested pass: 80/100 with no unbounded resource promise or unqualified reliability claim.

Module 45 — Compilers, shader languages and engine specialization

Prerequisites: Modules 15-16, 18, 37 and 41-42.

Original project brief: Gate I: deliver a research-quality reproduction and extension in rendering, simulation, GPU algorithms or compilation. This module also requires a small compiler optimization with semantic tests, generated-code inspection and end-to-end measurements.

Core reference IDs: R14, R11, R37, R48. Full entries are in the reading directory.

- 45.1 — Intermediate representations, SSA and dataflow analysis
- 45.2 — Alias analysis, vectorization and legal optimization
- 45.3 — Domain-specific languages, specialization and safe JIT execution
- 45.4 — Compiler/hardware co-design and graduate artifact review

Lesson 45.1 — Intermediate representations, SSA and dataflow analysis

Learning outcome: Represent program meaning so transformations can be justified.

Planning: 24-38 hours. Includes assigned study and practical work; module/Gate I time overlaps.

Original teaching primer

An intermediate representation makes control flow, values and operations explicit. Static single assignment gives each value one defining assignment; merge points use a representation such as phi nodes to select values from predecessor paths. SSA simplifies many analyses but does not by itself establish program correctness. Dataflow analysis computes properties over a control-flow graph, often through iterative fixed points on a chosen domain. Liveness identifies values that may be used later; constant propagation tracks values known under the analysis. Soundness means the analysis does not exclude behavior relevant to its guarantee, while precision controls how much useful information it retains. Dominance and control-flow edges must remain consistent after transformations.

SSA is an explicit representation of alternatives

In static single assignment form, each value name has one definition. A control-flow merge selects a value from a predecessor through a phi node. The choice follows the edge actually taken; it is not an eager evaluation of both branches. For a loop, a header phi can choose the initial value on entry and the updated value on the backedge. Dominance requires a definition to be available on every path to an ordinary use; phi inputs have edge-specific rules. [S13]

Changing control-flow edges requires updating phi predecessors and preserving dominance. A transformation that deletes a branch but leaves an obsolete phi input is not a harmless formatting issue. In a loop with multiple phis, all incoming values are selected from the predecessor environment together; assigning them sequentially can accidentally make one phi read another's new value rather than the old edge value.

Dataflow is a fixed-point computation over a chosen abstraction

For backward liveness, OUT[B] is the union of IN sets for successor blocks and IN[B]=USE[B] union (OUT[B] minus DEF[B]). Iterate until no set changes. On a finite variable universe, a monotone formulation starting from empty sets reaches a fixed point. This computes conservative possible future uses, not the exact concrete values of every execution.

In SSA, phi operands are uses on specific incoming edges. A generic block-level liveness routine without phi-aware edge treatment needs preprocessing or extension. The included live_variables function uses an explicit ordinary USE/DEF/successor input contract; do not silently feed it phi semantics it does not implement. Work-list scheduling changes the number of iterations, not the intended least fixed point of this monotone framework.

An interpreter is a differential-testing anchor

Define integer width, overflow, branch interpretation, unsupported operations and the step limit of the reference interpreter. The supplied CFG executes unsigned 32-bit arithmetic modulo 2^32 and has a fuel limit so a malformed or nonterminating student program does not run forever. It handles branches, loop phis and bounded execution, but does not statically prove unique definitions, full dominance or type consistency for every possible IR.

Before optimizing, add a verifier for the subset you accept. Then compare original and transformed programs over directed boundary values and randomized inputs. A concrete differential test can find a bug; it does not replace a legality argument for all inputs. Preserve both the IR before/after and a minimal failing input when the comparison disagrees.

Worked engineering example

Consider an if/else that yields a=1 on the then edge and b=2 on the else edge. The merge is x=phi(then:a, else:b). With a runtime condition, the abstract constant result is not known to be 1. If constant propagation proves the condition true, it may remove the else edge and replace the phi by a, after repairing the CFG. For liveness, if a later block uses x, x is live into that block; the corresponding incoming operand is live only on the edge that supplies it in a phi-aware analysis.

Original worked check

After an if/else assigning x=1 or x=2, a merge can represent x=phi(1 from then, 2 from else). Replacing it with 1 unconditionally would be wrong unless the else path is proven unreachable.

Practical assignment

Extend the tiny compiler with a control-flow IR and one dataflow analysis. Pass: derive transfer functions and a fixed-point stopping condition, test loops and merges, and verify transformed programs against an interpreter.

Full execution steps and acceptance checks are in LAB_WORKBOOK.pdf under this same lesson ID. Your warm-up file is `code/exercises/lesson_45_1.py`.

Reference starting point: code/gradlab/compiler.py: run_cfg, live_variables

Assigned study

Original assignment: R14: LLVM frontend, control flow and optimization tutorials; R48: compiler/architecture interaction.

Focused expansion: R14 and S13: LLVM control-flow and SSA semantics; use the original tiny compiler from Module 15 as the frontend where available. The supplied CFG interpreter is an executable semantic oracle, not a complete IR verifier.

Check your understanding

Attempt all three original questions before opening the key. A lesson also requires its reading, lab and graduate problems.

1. What does SSA require of each value?
2. Why is the example phi node not always constant 1?
3. What is the role of a fixed point in dataflow analysis?

Lesson 45.2 — Alias analysis, vectorization and legal optimization

Learning outcome: Separate transformations that are fast from transformations that preserve meaning.

Planning: 22-34 hours. Includes assigned study and practical work; module/Gate I time overlaps.

Original teaching primer

An optimization must preserve the language's permitted observable behavior under its assumptions. Alias analysis determines which references may identify overlapping storage. Incorrectly assuming no aliasing can make loop reordering or vectorization wrong. Undefined behavior, overflow rules and floating-point options affect what transformations are legal, so record the compilation contract. Vectorization groups iterations only when dependencies and semantics permit it. Reductions, exceptional values and rounding can require special handling. Register allocation maps values to a limited physical register set; spills introduce memory traffic. Inspect generated code and test adversarial inputs. A faster compiler output that silently changes required results is a correctness failure, not an optimization.

Legality comes before profitability

A transformation is legal only if it preserves the permitted observable behavior under its declared language contract. For unsigned 32-bit arithmetic, additions and multiplications wrap modulo 2^32. Algebraic identities valid in that ring can justify constant folding. Signed overflow, floating-point reassociation and exception behavior obey different rules; do not reuse a proof across those contracts without checking the assumptions.

The reference native example checks (x+0)*(2+3) against x*5* for 100,000 unsigned inputs and has a simple algebraic justification. Those tests support implementation confidence; the ring identity supplies the all-input argument. Converting the example to a floating-point expression with NaN, infinities or signed zero can invalidate identities that seemed obvious over real numbers. [S13]

Alias and dependence facts constrain the schedule

Two syntactically different references can denote overlapping storage. A write through one reference may change what a later read through another observes. Before reordering, distinguish true data dependence, anti-dependence and output dependence. A loop-carried recurrence such as a[i]=a[i-1]+1 cannot simply be evaluated as independent lanes.

For half-open logical ranges [a,a+n) and [b,b+m), overlap exists when a<b+m and b<a+n, provided both lengths are positive. The reference models integer offsets. Native pointer ordering/provenance and overflow-safe address arithmetic require additional language-specific care. A runtime range check can dispatch to a vectorized no-overlap path and a conservative fallback, but a false no-alias annotation does not make overlapping inputs legal. [S15]

Inspect the complete generated computation

Request optimization remarks and inspect generated assembly or intermediate code for the actual target. An optimization may add a runtime alias guard, scalar remainder loop, gathers, conversions or spills. Count the effect on the calling workload, not just one attractive vector instruction. Short loops may spend more time dispatching than computing.

Keep semantic tests active in optimized builds. The native reference uses explicit checks rather than assertions that disappear under common Release settings. Run sanitizers separately from timing; instrumentation changes execution cost. Sweep aligned, unaligned, overlapping where permitted, empty and remainder sizes. Reject any speed result that violates the required numerical or memory contract.

Worked engineering example

Let a initially be [7,0,0,0] and apply a[i]=a[i-1]+1 for i=1,2,3 in order. The result is [7,8,9,10]. A naive simultaneous implementation that reads all original previous elements gives [7,8,1,1]. There may be a valid alternative algorithm for this particular recurrence, but proving and implementing that algorithm is different from asserting that the original iterations are independent. The adversarial initial values make the hidden dependence obvious.

Original worked check

For a[i]=a[i-1]+1, iterations depend on previous results. Treating all iterations as independent changes behavior unless a valid alternative algorithm preserves the recurrence.

Practical assignment

Implement or study a loop transformation with an explicit legality check. Pass: overlapping-memory and boundary cases are tested, compiler flags are recorded and speed claims exclude transformations that violate the required numerical contract.

Full execution steps and acceptance checks are in LAB_WORKBOOK.pdf under this same lesson ID. Your warm-up file is `code/exercises/lesson_45_2.py`.

Reference starting point: code/gradlab/compiler.py: ranges_overlap, fold; code/native/main.cpp

Assigned study

Original assignment: R14: optimization and code generation; R11: aliasing, lifetime and arithmetic guidance; R37: vector/parallel execution consequences.

Focused expansion: S13: LLVM language semantics; S15: LLVM loop and SLP vectorizers; R11: lifetime, arithmetic and aliasing contracts. Record actual compiler versions and flags, not an assumed universal optimizer behavior.

Check your understanding

Attempt all three original questions before opening the key. A lesson also requires its reading, lab and graduate problems.

1. Why is alias analysis important for reordering loads and stores?
2. Are iterations of a[i]=a[i-1]+1 independent?
3. Why record floating-point compiler options?

Lesson 45.3 — Domain-specific languages, specialization and safe JIT execution

Learning outcome: Trade generality for predictable game-engine computation.

Planning: 22-34 hours. Includes assigned study and practical work; module/Gate I time overlaps.

Original teaching primer

A domain-specific language restricts expression to make authoring, validation or optimization easier. A material graph, particle formula or animation expression can expose structure that a general compiler cannot easily infer. Define types, allowed effects and resource limits before implementing code generation. Specialization precomputes decisions known from material or scene parameters, potentially reducing runtime branches at the cost of more variants and compilation work. Just-in-time compilation introduces startup latency, code-cache management and execution-security concerns. Cache keys must include meaningful inputs and tool versions. Untrusted scripts or graphs need capability and complexity limits. A restricted interpreter remains a useful fallback and semantic reference.

Restrict the language to make its contract testable

A useful game-engine DSL may express a material formula, particle update or animation parameter rather than an arbitrary program. Define its types, effects, evaluation order and resource limits. The supplied expression language accepts unsigned 32-bit constants, named inputs, addition and multiplication. It forbids filesystem, network, calls, dynamic imports and arbitrary evaluation. Trees are validated before interpretation or code generation.

Node-count and depth limits bound the work of this tree interpreter. Inputs are range-checked. Generated input names come from a controlled identifier mapping rather than pasting user strings directly into C++ source. These controls reduce the accepted language's risk and simplify optimization, but they are not an operating-system isolation boundary. Untrusted native code still needs a separate security design.

Specialization trades runtime work for variants

A constant material option can remove a runtime branch or fold a subexpression. With k independent binary options, there can be 2^k combinations before considering formats, lighting modes or targets. Not every combination needs eager compilation. Use a bounded cache, compilation budget, prioritized common variants and a reference fallback for misses.

The cache key must include all semantic inputs: source graph, specialization parameters, target, arithmetic flags, ABI/schema and relevant compiler version. Omitting a target feature or floating-point option can make an old binary appear valid under a new contract. Record cache hits, misses, compile latency, peak bytes and eviction behavior rather than measuring only warmed execution.

Keep code generation separate from permission to execute

The interpreter is the semantic reference and portable fallback. Constant folding must validate its input and preserve the unsigned arithmetic contract. Generated C++ should be inspected and tested against the interpreter for a bounded trusted corpus. Native compilation is an optional extension; no raw graph should cause the package to invoke a shell with user-controlled text.

A real JIT must additionally manage executable memory, code lifetime during concurrent use, cancellation, target capabilities and isolation. Writable/executable memory policies are one part of the design, not a complete sandbox. Process boundaries, capabilities, resource quotas and trusted compiler/runtime components may be needed depending on the threat model. The supplied code deliberately stops at restricted interpretation and source generation so it does not imply those protections have been implemented. [S16]

Worked engineering example

Eight binary material options allow 256 variants. At 80 KiB of code per variant, eagerly retaining all of them consumes 20,480 KiB, exactly 20 MiB, before metadata or other target variants. At 12 ms of compilation per variant, serial eager compilation adds 3.072 seconds. Specializing only the 20 common combinations initially reduces the immediate code budget to about 1.56 MiB and compilation to 240 ms, while uncommon cases use the fallback or a bounded later compilation path. Whether this improves the game depends on actual option frequencies and latency constraints.

Original worked check

Specializing four binary material options can create up to 2^4=16 variants before other parameters are considered. Uncontrolled variant growth can increase build time and runtime cache misses.

Practical assignment

Build a typed expression or material DSL with constant folding and a bounded interpreter or JIT route. Pass: invalid graphs fail safely, variant counts are measured and compiled outputs match the reference.

Full execution steps and acceptance checks are in LAB_WORKBOOK.pdf under this same lesson ID. Your warm-up file is `code/exercises/lesson_45_3.py`.

Reference starting point: code/gradlab/compiler.py: validate_expr, interpret, fold, cache_key, generate_cpp

Assigned study

Original assignment: R14: language implementation and JIT; R26: material and scripting concepts for comparison; Module 20: trust boundaries.

Focused expansion: R14 / S16: LLVM language and JIT tutorials; S13: generated-code semantics; R26: engine material and scripting interfaces. This package emits inspectable C++ but does not execute an untrusted native JIT.

Check your understanding

Attempt all three original questions before opening the key. A lesson also requires its reading, lab and graduate problems.

1. How many combinations do four independent binary options create?
2. Why limit the effects allowed in a DSL?
3. Why keep an interpreter alongside compiled execution?

Lesson 45.4 — Compiler/hardware co-design and graduate artifact review

Learning outcome: Connect a transformation to architecture and application evidence.

Planning: 22-34 hours plus overlapping Gate I integration. Includes assigned study and practical work; module/Gate I time overlaps.

Original teaching primer

Compiler decisions influence memory layout, instruction mix, register demand and parallel scheduling. Hardware features are useful only when software can express and exploit them efficiently. A co-design experiment changes one interface, representation or instruction opportunity and measures its complete effect. Avoid attributing a gain to hardware when it actually comes from a stronger software baseline. Graduate review requires a defensible claim, mathematical or semantic argument, implementation and reproducible evaluation. Gate I can focus on one earlier graduate topic, but must include baselines, ablations, uncertainty and limitations. Publication acceptance is not required; credible independent evaluation is. Preserve unsuccessful experiments and distinguish a reproduced known result from the learner's actual extension.

Explain the mechanism across the software/hardware boundary

A compiler transformation can change instruction count, data layout, register pressure, cache behavior and parallel occupancy at once. Attribute a claimed gain to evidence, not to the name of an instruction or architecture. A wider vector operation may require packing, unpacking, alignment handling and scalar tails. Saving arithmetic can increase memory traffic or reduce the number of resident workers.

Build a cost decomposition that links the generated code to the application's critical path. Compare a competent software baseline before proposing an accelerator or new instruction. An ablation should remove the proposed mechanism while keeping unrelated improvements fixed. A sensitivity study should vary a parameter that can expose the mechanism's limits, such as batch size, arithmetic intensity or register demand.

Complete one coherent research argument

Gate I requires a bounded reproduction plus a clearly identified extension, new setting or rigorous failure analysis. State exactly what prior result you are testing, which assumptions you retained and how your comparison differs. A reproduction discrepancy can become useful evidence when investigated carefully; it is not permission to invent a matching result or silently retune the baseline.

Deliver the derivation or semantic proof, code, workload definitions, raw data, analysis scripts, one ablation, one sensitivity study and a limitations section. Use equal-quality comparisons when methods change quality with speed. Identify the trial unit behind uncertainty estimates. Preserve negative cases and the tuning/evaluation split. This work prepares the thesis; it does not need to establish global novelty or publication acceptance.

The reviewer must be able to challenge the central claim

An independent reader should build or run the artifact and regenerate its main table from raw data. Ask the reader to inspect one failed case, alter one input and explain the legality or numerical assumptions behind the main transformation. Record discrepancies and revisions. A README that says results are reproducible is not evidence that another person actually reproduced them. [S14]

Even when Gate I focuses on rendering or simulation, this module still requires the small compiler optimization with semantic tests and generated-code inspection. The volume's breadth exercises support the specialization rather than disappearing once a favorite topic is chosen. Completion is recorded as demonstrated self-study work, with pending GPU, hardware or independent-review evidence marked honestly.

Worked engineering example

An engine frame takes 12 ms: 3 ms in a candidate kernel and 9 ms elsewhere. A proposed instruction halves the kernel arithmetic, apparently saving 1.5 ms. But the new layout adds 0.8 ms of packing and 0.9 ms of extra spill/transfer work. The new frame takes 9+1.5+0.8+0.9=12.2 ms, so speedup is 12/12.2, approximately 0.984. The kernel-only 2x number is real under its narrow boundary, but the integrated change is a regression. A sound report can conclude that the instruction is unattractive for this workload without treating the experiment as a failure of learning.

Original worked check

A new vector instruction that saves arithmetic but increases packing and spills may lose in the full engine. The experiment must measure both the generated kernel and its application integration.

Practical assignment

Complete Gate I and defend one compiler transformation's legality. Pass: an independent reader reruns the central experiment, the claimed extension is precisely identified and raw evidence supports the conclusions.

Full execution steps and acceptance checks are in LAB_WORKBOOK.pdf under this same lesson ID. Your warm-up file is `code/exercises/lesson_45_4.py`.

Reference starting point: code/run_experiments.py; code/native/main.cpp; templates/INDEPENDENT_REVIEW.md

Assigned study

Original assignment: R14 and R48: compilation and architecture; R49: co-design methodology; Module 41: research evaluation protocol.

Focused expansion: R14, R48 and R49: compiler/architecture interfaces and co-design; S14: artifact review principles. Gate I uses this curriculum's own rubric and does not award an ACM badge or an accredited degree.

Check your understanding

Attempt all three original questions before opening the key. A lesson also requires its reading, lab and graduate problems.

1. Why can a faster instruction fail to improve the application?
2. What distinguishes reproduction from extension?
3. Is publication acceptance mandatory for Gate I?

Module 45 project: compiler mini-project plus Gate I

The compiler requirement remains even when Gate I specializes elsewhere. Extend the prior tiny compiler or the supplied restricted reference with a verified IR subset and a dataflow/optimization pass. Cover loops and merges, state phi/alias/arithmetic semantics, and compare transformed execution against an interpreter. Inspect actual generated code and measure a bounded integration workload, not only expression evaluation in isolation.

Required artifacts: accepted language/IR contract, verifier checks, transfer functions or legality proof, original/transformed IR, directed and randomized tests, compiler flags, generated-code inspection and end-to-end measurements. Semantics 40 points, analysis/verifier 25, generated-code/performance evaluation 20, artifact/limitations 15. Suggested pass: 80/100 with no semantic mismatch. An existing constant-folding smoke test alone does not satisfy this project.

Complete Gate I using CAPSTONE_SPEC.pdf. The compiler mini-project and Gate I can overlap when compilation is the chosen depth area; otherwise they are distinct requirements within the volume's planning estimate. Independent review is required for Gate I, not silently replaced by the reference test suite.

Primary reading directory

R01-R55 retain the original program IDs. S01-S17 are focused additions for this expansion. Citations such as [S05] refer to the entries below; equations and worked cases not attributed to another source are original teaching derivations. External resources retain their own rights and are linked rather than redistributed. Moving documentation is not a promise of compatibility with an untested installed version. Record exact versions for your own artifact.

R07 — 18.06 Linear Algebra

MIT OpenCourseWare

https://ocw.mit.edu/courses/18-06-linear-algebra-spring-2010/

Matrices, vector spaces, eigenvalues and problem sets.

Source status: Inherited from the original program; not every moving page was revalidated for this expansion.

R09 — 18.05 Introduction to Probability and Statistics

MIT OpenCourseWare

https://ocw.mit.edu/courses/18-05-introduction-to-probability-and-statistics-spring-2022/

Probability, inference and quantitative reasoning.

Source status: Inherited from the original program; not every moving page was revalidated for this expansion.

R11 — C++ Core Guidelines

C++ Core Guidelines editors

https://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines

Resource safety, interfaces, classes and concurrency; guidance rather than the normative language standard.

Source status: Inherited from the original program; not every moving page was revalidated for this expansion.

R14 — Kaleidoscope and LLVM tutorials

LLVM Project

https://llvm.org/docs/tutorial/

Compiler frontends, intermediate representation and JIT compilation.

Source status: Inherited from the original program; not every moving page was revalidated for this expansion.

R17 — RFC 9293: Transmission Control Protocol

IETF / RFC Editor

https://www.rfc-editor.org/rfc/rfc9293.html

Transport semantics; reference reading after networking foundations.

Source status: Inherited from the original program; not every moving page was revalidated for this expansion.

R18 — PostgreSQL tutorial

PostgreSQL Global Development Group

https://www.postgresql.org/docs/current/tutorial.html

Relational data, transactions and SQL; pin local server and client versions.

Source status: Inherited from the original program; not every moving page was revalidated for this expansion.

R20 — 15-466/666 Computer Game Programming

Carnegie Mellon University

https://graphics.cs.cmu.edu/courses/15-466/

Runtime systems and game-production programming.

Source status: Inherited from the original program; not every moving page was revalidated for this expansion.

R21 — Vulkan tutorial

Khronos Group

https://docs.vulkan.org/tutorial/latest/00_Introduction.html

Explicit graphics API study; feature-check the device before using the moving latest tutorial.

Source status: Inherited from the original program; not every moving page was revalidated for this expansion.

R23 — Physically Based Rendering, fourth edition

Matt Pharr, Wenzel Jakob, Greg Humphreys

https://www.pbr-book.org/4ed/contents

Rendering mathematics and implementation; use chapter titles in lesson assignments.

Source status: Inherited from the original program; not every moving page was revalidated for this expansion.

R24 — Direct3D 12 graphics

Microsoft

https://learn.microsoft.com/en-us/windows/win32/direct3d12/direct3d-12-graphics

Windows graphics backend reference.

Source status: Inherited from the original program; not every moving page was revalidated for this expansion.

R25 — Metal developer resources

Apple

https://developer.apple.com/metal/

Apple-platform graphics and compute reference.

Source status: Inherited from the original program; not every moving page was revalidated for this expansion.

R26 — Programming with C++ in Unreal Engine

Epic Games

https://dev.epicgames.com/documentation/en-us/unreal-engine/programming-with-cplusplus-in-unreal-engine

Engine comparison after C++ foundations, not a beginner substitute.

Source status: Inherited from the original program; not every moving page was revalidated for this expansion.

R37 — CUDA Programming Guide

NVIDIA

https://docs.nvidia.com/cuda/cuda-programming-guide/

GPU execution and memory; CUDA is an optional vendor-specific lab route.

Source status: Inherited from the original program; not every moving page was revalidated for this expansion.

R41 — 18.02SC Multivariable Calculus

MIT OpenCourseWare

https://ocw.mit.edu/courses/18-02sc-multivariable-calculus-fall-2010/

Partial derivatives, gradients, multiple integrals and constrained differentials.

Source status: Inherited from the original program; not every moving page was revalidated for this expansion.

R42 — Convex Optimization

Stephen Boyd and Lieven Vandenberghe

https://web.stanford.edu/~boyd/cvxbook/

Graduate optimization text available from its authors; copyrighted, linked rather than redistributed.

Source status: Inherited from the original program; not every moving page was revalidated for this expansion.

R43 — Roofline performance model, technical report

Samuel Williams, Andrew Waterman and David Patterson

https://www2.eecs.berkeley.edu/Pubs/TechRpts/2008/EECS-2008-134.html

Original 2008 performance-model reference; use current measured hardware limits for experiments.

Source status: Inherited from the original program; not every moving page was revalidated for this expansion.

R44 — Spatiotemporal reservoir resampling for real-time ray tracing with dynamic direct lighting

Benedikt Bitterli and coauthors

https://research.nvidia.com/publication/2020-07_spatiotemporal-reservoir-resampling-real-time-ray-tracing-dynamic-direct

Original ReSTIR research, 2020; example reproduction target, not a claim to be the latest method.

Source status: Inherited from the original program; not every moving page was revalidated for this expansion.

R47 — Dive into Deep Learning

Aston Zhang, Zachary Lipton, Mu Li and Alexander Smola

https://d2l.ai/

Learning fundamentals, gradients, generalization and reproducible exercises; pin compatible framework versions.

Source status: Inherited from the original program; not every moving page was revalidated for this expansion.

R48 — 6.823 Computer System Architecture

MIT OpenCourseWare

https://ocw.mit.edu/courses/6-823-computer-system-architecture-fall-2005/

Graduate architecture foundations; historical course concepts supplemented with current specifications.

Source status: Inherited from the original program; not every moving page was revalidated for this expansion.

R49 — ECE5745 Complex Digital ASIC Design

Christopher Batten / Cornell University

https://www.csl.cornell.edu/courses/ece5745/

Graduate ASIC methodology, circuits, EDA and design-project expectations.

Source status: Inherited from the original program; not every moving page was revalidated for this expansion.

R50 — SymbiYosys documentation

YosysHQ

https://yosyshq.readthedocs.io/projects/sby/en/latest/

Formal hardware verification, safety, liveness and cover properties.

Source status: Inherited from the original program; not every moving page was revalidated for this expansion.

R54 — 18.085 Computational Science and Engineering I

MIT OpenCourseWare

https://ocw.mit.edu/courses/18-085-computational-science-and-engineering-i-fall-2008/

Graduate numerical methods, equilibrium systems, differential equations and variational reasoning.

Source status: Inherited from the original program; not every moving page was revalidated for this expansion.

R55 — Author publication archive: position-based simulation and XPBD

Matthias Mueller and collaborators

https://matthias-research.github.io/pages/publications/publications.html

Read the named 2016 XPBD paper and related course notes for the simulation reproduction; papers retain their own copyrights.

Source status: Inherited from the original program; not every moving page was revalidated for this expansion.

S01 — An Introduction to the Conjugate Gradient Method Without the Agonizing Pain (1994)

Jonathan Richard Shewchuk

https://www.cs.cmu.edu/~quake-papers/painless-conjugate-gradient.pdf

PCG derivation and Appendix B3; PDF and algorithm page inspected.

Source status: Focused primary source consulted September 28, 2026; see note for retrieval limits.

S02 — Convex Optimization (2004), author-hosted text

Stephen Boyd and Lieven Vandenberghe

https://web.stanford.edu/~boyd/cvxbook/bv_cvxbook.pdf

Convexity, optimality and numerical methods; KKT page inspected. Also R42.

Source status: Focused primary source consulted September 28, 2026; see note for retrieval limits.

S03 — Parallel Prefix Sum (Scan) with CUDA, GPU Gems 3, Chapter 39

Mark Harris, Shubhabrata Sengupta and John D. Owens

https://developer.nvidia.com/gpugems/gpugems3/part-vi-gpu-computing/chapter-39-parallel-prefix-sum-scan-cuda

Work-efficient scan foundations; historical algorithms, not current tuning defaults.

Source status: Focused primary source consulted September 28, 2026; see note for retrieval limits.

S04 — CUDA Programming Guide

NVIDIA

https://docs.nvidia.com/cuda/cuda-programming-guide/

Execution and memory contracts; moving documentation consulted, no matching toolkit/device validation claimed.

Source status: Focused primary source consulted September 28, 2026; see note for retrieval limits.

S05 — XPBD: Position-Based Simulation of Compliant Constrained Dynamics (2016)

Miles Macklin, Matthias Mueller and Nuttapong Chentanez

https://matthias-research.github.io/pages/publications/XPBD.pdf

Original paper; multiplier/compliance equations inspected. A bounded reproduction target, not a completed result supplied here.

Source status: Focused primary source consulted September 28, 2026; see note for retrieval limits.

S06 — PBRT 4e: Mapping Path Tracing to the GPU

Matt Pharr, Wenzel Jakob and Greg Humphreys

https://www.pbr-book.org/4ed/Wavefront_Rendering_on_GPUs/Mapping_Path_Tracing_to_the_GPU

Wavefront organization and queue-oriented rendering.

Source status: Focused primary source consulted September 28, 2026; see note for retrieval limits.

S07 — PBRT 4e: Improving Efficiency

Matt Pharr, Wenzel Jakob and Greg Humphreys

https://www.pbr-book.org/4ed/Monte_Carlo_Integration/Improving_Efficiency

Importance sampling, MIS and roulette.

Source status: Focused primary source consulted September 28, 2026; see note for retrieval limits.

S08 — PBRT 4e: A Better Path Tracer

Matt Pharr, Wenzel Jakob and Greg Humphreys

https://www.pbr-book.org/4ed/Light_Transport_I_Surface_Reflection/A_Better_Path_Tracer

Light sampling and emitter-hit weighting; compare against the bounded educational implementation.

Source status: Focused primary source consulted September 28, 2026; see note for retrieval limits.

S09 — Vulkan specification: Acceleration Structures

Khronos Group

https://docs.vulkan.org/spec/latest/chapters/accelstructures.html

Capability, update and resource contracts. Moving specification, not a tested API backend.

Source status: Focused primary source consulted September 28, 2026; see note for retrieval limits.

S10 — Gradient-based optimization tutorial

Mitsuba project

https://mitsuba.readthedocs.io/en/stable/src/inverse_rendering/gradient_based_opt.html

Primary implementation documentation for inverse rendering; no Mitsuba installation required by the portable references.

Source status: Focused primary source consulted September 28, 2026; see note for retrieval limits.

S11 — In Search of an Understandable Consensus Algorithm, extended version

Diego Ongaro and John Ousterhout

https://raft.github.io/raft.pdf

Replicated state-machine assumptions; PDF and architecture page inspected. The teaching sink does not implement Raft.

Source status: Focused primary source consulted September 28, 2026; see note for retrieval limits.

S12 — Safety, Liveness, and Fairness (2019)

Leslie Lamport

https://lamport.azurewebsites.net/tla/safety-liveness.pdf

Precise definitions and fairness assumptions; safety/liveness page inspected.

Source status: Focused primary source consulted September 28, 2026; see note for retrieval limits.

S13 — LLVM Language Reference Manual

LLVM Project

https://llvm.org/docs/LangRef.html

SSA, phi and arithmetic semantics. Record the LLVM version actually used in a learner experiment.

Source status: Focused primary source consulted September 28, 2026; see note for retrieval limits.

S14 — Artifact Review and Badging, version 1.1

ACM

https://www.acm.org/publications/policies/artifact-review-and-badging-current

Official search index and ACM terminology page verified; direct policy-page fetch returned 403. Supplement: https://www.acm.org/publications/badging-terms . This curriculum awards no ACM badge.

Source status: Focused primary source consulted September 28, 2026; see note for retrieval limits.

S15 — Auto-Vectorization in LLVM

LLVM Project

https://llvm.org/docs/Vectorizers.html

Loop/SLP vectorization, legality and diagnostics; primary compiler documentation.

Source status: Focused primary source consulted September 28, 2026; see note for retrieval limits.

S16 — LLVM Tutorial: Table of Contents

LLVM Project

https://llvm.org/docs/tutorial/

Frontend, IR and JIT tutorials. The bundled DSL stops at interpretation and source emission.

Source status: Focused primary source consulted September 28, 2026; see note for retrieval limits.

S17 — Consul session semantics

HashiCorp

https://developer.hashicorp.com/consul/docs/automate/session

Session/locking behavior and authority limitations; primary service documentation, not a service deployment requirement.

Source status: Focused primary source consulted September 28, 2026; see note for retrieval limits.